

Dinámica y movimiento para videojuegos y gamificación

Tema 1. Conceptos Básicos

Pedro Ángel Bernaola Galván

Febrero 2026

Contenido

- 1 Introducción
- 2 Unidades de medida
- 3 Representación gráfica en 3D
- 4 Vectores
- 5 Matrices
- 6 Derivación numérica
- 7 Integración numérica
- 8 Resolución numérica de ecuaciones diferenciales

1 Introducción

La Física en los videojuegos

La Física sostiene la credibilidad e inmersión de los mundos virtuales y convierte una simple animación en una experiencia convincente.

- **Evita que “parezcan falsos”.** Los jugadores esperan comportamientos coherentes con la realidad: pilotar, lanzar una pelota o mover un personaje debe parecer correcto correcto.
- **El realismo es más fácil de lo que parece.** Predominan ecuaciones algebraicas y trigonométricas; los casos complejos (fluidos, aerodinámica) usan EDOs resueltas numéricamente.
- **No penaliza el rendimiento.** El coste dominante suele ser el *rendering*, no la física; los modelos razonables se ejecutan sin problemas en hardware actual.
- **Mejora tus habilidades como desarrollador.** Entender los fundamentos físicos guía qué efectos incluir y cómo implementarlos con eficiencia (ej.: efecto de una bola de golf, fricción, gravedad, colisiones).

¿Para qué necesito saber Física si ya existen motores físicos?

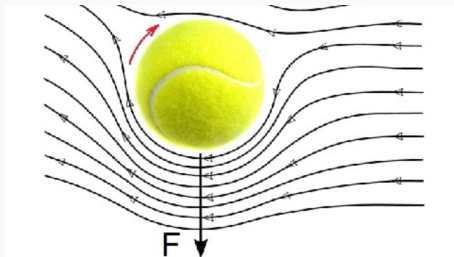
Los motores físicos (*Unity Physics*, *Unreal Chaos*, *Box2D*, *Bullet*, *Chipmunk2D* etc.) resuelven automáticamente colisiones, fuerzas, integración, contactos, fricción, etc. Pero **no toman decisiones de diseño**: solo ejecutan lo que tú les pides.

¿Por qué necesitas saber Física entonces?

- **Para ajustar el comportamiento**: masa, amortiguamiento, fricción, rebote... Sin entender su fundamento físico, es imposible afinarlos correctamente.
- **Para detectar y resolver problemas**: jitter, penetración, explosiones numéricas, cuerpos que salen disparados, acumulación de energía, errores de integración...
- **Para saber qué motor usar y cómo configurarlo**: raycasts, rigid bodies, constraints, joints, integradores, capas de colisión, etc.

¿Para qué necesito saber Física si ya existen motores físicos?

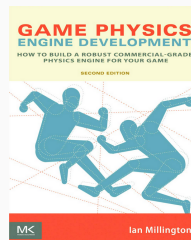
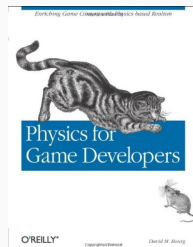
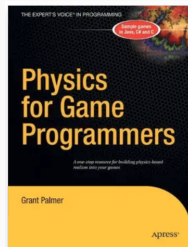
- **Para crear efectos que el motor no trae “de fábrica”:** viento, fuerzas aerodinámicas, balística realista, partículas físicas, ragdolls creíbles...
- **Para no depender ciegamente del motor:** los juegos buenos requieren **control**, no milagros.



Referencias recomendadas

Libros de Física

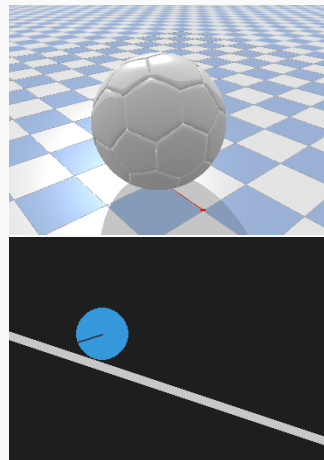
- Palmer, Grant. *Physics for Game Programmers*. Apress 1st. ed., 2005
- Bourg, David M. *Physics for Game Developers*. O'Reilly Media, 2002.
- Millington, Ian. *Game Physics Engine Development*. CRC Press, 2010.



Referencias recomendadas

Physics engines

- **Bullet Physics SDK** Erwin Coumans. GitHub repository.
<https://github.com/bulletphysics/bullet3>
- **Chipmunk** Scott Lembcke. *Chipmunk2D: A fast and lightweight 2D game physics library.*
Version 7.0.3, MIT License, 2019.
<https://github.com/slembcke/Chipmunk2D>
- Aquí usaremos wrappers para Python (al menos yo)
 - Pybullet para Bullet
<https://pypi.org/project/pybullet/>
 - Pymunk para Chipmunk
<https://pypi.org/project/pymunk/>



2 Unidades de medida

Unidades de medida

- Para describir el mundo físico, lo que hacemos es medir las distintas magnitudes de interés (masa, velocidad, etc).
- Para medir cada tipo de magnitud, necesitamos una **unidad**.
- Normalmente, se toman como magnitudes fundamentales la longitud $[L]$, la masa $[M]$ y el tiempo $[T]$. El resto de magnitudes se derivan a partir de estas. A la dependencia de una magnitud derivada con respecto a las fundamentales se le llama las **dimensiones** de la unidad derivada. **Ejemplo:** la velocidad tiene las dimensiones $[L]/[T]$.
- Al conjunto formado por las unidades usadas para medir las distintas magnitudes se le llama **Sistema de unidades**. Nosotros usaremos el **Sistema Internacional (S.I.)**.
- El S.I. también se llama **MKS**, porque las unidades de las magnitudes fundamentales son el metro, el kilogramo y el segundo.

Sistema Internacional

Magnitud	Dim.	S.I.
Longitud	$[L]$	m
Tiempo	$[T]$	s
Masa	$[M]$	kg
Velocidad	$[L][T]^{-1}$	m/s
Aceleración	$[L][T]^{-2}$	m/s ²
Fuerza	$[M][L][T]^{-2}$	N
Presión	$[M][L]^{-1}[T]^{-2}$	Pa

Magnitud	Dim.	S.I.
Energía	$[M][L]^2[T]^{-2}$	J
Potencia	$[M][L]^2[T]^{-3}$	W
Superficie	$[L]^2$	m ²
Volumen	$[L]^3$	m ³
Densidad	$[M][L]^{-3}$	kg/m ³
Cant. movimiento	$[M][L][T]^{-1}$	kg·m/s
Impulso	$[M][L][T]^{-1}$	N·s
Frecuencia	$[T]^{-1}$	Hz

3 Representación gráfica en 3D

Sistemas de coordenadas

- En juegos y simulaciones, gran parte de la física se reduce a **posicionar** y **mover** objetos en el espacio.
- Usaremos **coordenadas cartesianas** por defecto y **esféricas** cuando convenga (cámaras/orbitas).
- Trabajaremos con **escalares** y **vectores**; veremos sus **operaciones** básicas y dos productos clave: **producto escalar** y **producto vectorial**.

Sistemas de coordenadas

Cartesianas (x, y, z)

- Convención típica en juegos: ejes x e y paralelos al suelo; z hacia arriba.
- Fáciles de visualizar y de integrar con motores de render.

Esféricas (r, θ, φ)

- Útiles en órbitas y cámaras “alrededor” de un objetivo.
- r : **Distancia al origen** (radial)
- θ : **Ángulo cenital** (desde $+Z$), $\theta \in [0, \pi)$
- φ : **Azimut en el plano XY** desde $+x$, $\varphi \in [0, 2\pi]$.

Ejemplos de coordenadas esféricas en videojuegos

- Cámara en tercera persona orbitando al personaje.
- Posición de un enemigo en “radar” (distancia y ángulos).
- proyectiles con trayectoria circular/helicoidal.

Conversión entre coordenadas esféricas y cartesianas

De esféricas a cartesianas

$$x = r \sin \theta \cos \varphi$$

$$y = r \sin \theta \sin \varphi$$

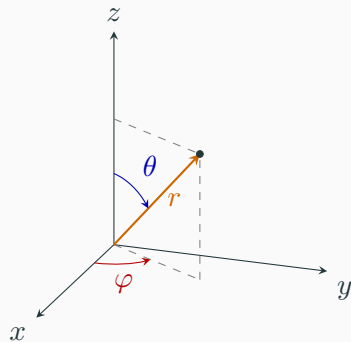
$$z = r \cos \theta$$

De cartesianas a esféricas

Distancia radial $r = \sqrt{x^2 + y^2 + z^2}$

Ángulo cenital $\theta = \arccos\left(\frac{z}{r}\right)$

Ángulo acimutal $\varphi = \text{atan2}(y, x)$



Convención: θ desde eje z (polar)

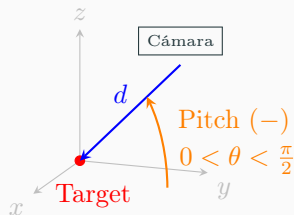
La cámara en PyBullet

- En la práctica, los motores físicos como **PyBullet** usan una variante de las coordenadas esféricas para controlar la cámara.
- `cameraDistance` (d): Igual que r . Es la distancia de la cámara al punto de observación.
- `cameraYaw` (ψ): Similar a φ pero $\psi = 0$ cuando la cámara está sobre el eje Y negativo y mira hacia el eje Y positivo:
 - $\psi = \varphi + \pi/2$ por lo que también crece en sentido antihorario
 - $\psi \in (-\pi, \pi]$
- `cameraPitch` (α): Similar a θ pero se mide desde el plano XY
 - $\alpha = \theta - \pi/2$. Es negativo por encima del plano y positivo por debajo.
 - $\alpha \in [-\pi/2, \pi/2]$

La cámara en PyBullet: Pitch

Interpretación de la Cámara

- Estas coordenadas corresponden a la posición relativa de la cámara con respecto al objeto. Si la cámara está arriba, ve al objeto abajo.
- **Pitch (-) = Mirar abajo:** Cuando la cámara está arriba, $\theta \in [0, \pi/2]$, el Pitch es negativo
 $\alpha = \theta - \pi/2 \in [-\pi/2, 0]$. Al aumentar el Pitch, la cámara sube en altura sobre el horizonte. Para mantener el *Target* centrado, la lente debe inclinarse hacia abajo.
- **Distancia (d):** Es el radio de la esfera de orbitación.



El sistema no pregunta **¿Dónde está el objeto?**,
 sino **¿Desde dónde estoy grabando al objeto?**

La cámara en PyBullet. Conversión entre coordenadas.

IMPORTANTE recordar que hablamos de las coordenadas de la posición de la cámara, relativa al objeto.

De PyBullet a cartesianas

$$x = d \cos \alpha \sin \psi$$

$$y = -d \cos \alpha \cos \psi$$

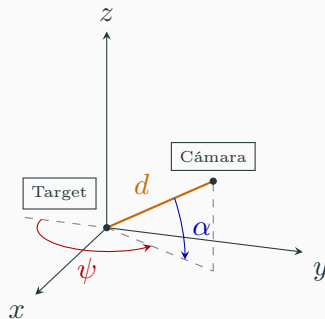
$$z = -d \sin \alpha$$

De cartesianas a PyBullet

Distance $d = \sqrt{x^2 + y^2 + z^2}$

Pitch $\alpha = -\arcsin\left(\frac{z}{d}\right)$

Yaw $\psi = \text{atan2}(x, -y)$



Convención PyBullet: α (Pitch) desde plano XY y ψ (Yaw) desde eje $-Y$.

La cámara en PyBullet. Unidades

¡Ojo con las unidades de medida!

Contexto	Ángulos	Funciones
Cálculo físico de vectores, velocidades angulares, etc.	Radianes	<code>np.sin()</code> , <code>np.cos()</code> , etc.
Visualización PyBullet	Grados	<code>resetDebugVisualizerCamera()</code>

- Para convertir: $\text{grados} = \text{radianes} \times 180/\pi$

Implementación en PyBullet

```
1  import pybullet as p
2  import pybullet_data
3  import time
4  # Configuración inicial
5  p.connect(p.GUI)
6  p.setAdditionalSearchPath(pybullet_data.getDataPath())
7  p.setGravity(0, 0, -9.81)
8  # Cargar suelo y cubo pequeño
9  p.loadURDF("plane.urdf")
10 cubo_id = p.loadURDF("cube_small.urdf", [0, 0, 0.03])
11 # Ajustar cámara: yaw=180 mira hacia Y- desde Y+
12 p.resetDebugVisualizerCamera(cameraDistance=0.173, cameraYaw=180, \
13     cameraPitch=-45, cameraTargetPosition=[0, 0, 0])
14 # Bucle de simulación
15 while True:
16     p.stepSimulation()
17     time.sleep(1./240.)
18
```


4 Vectores

Magnitudes escalares y vectoriales

Escalares

- Quedan definidos por un único número (masa, tiempo, temperatura).
- Operan con aritmética usual.

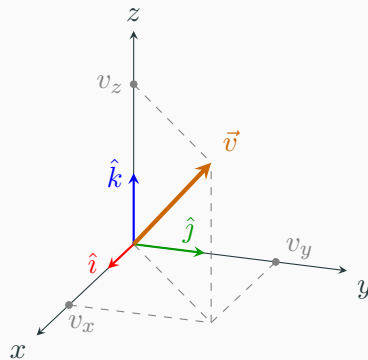
Vectores

- Tienen **módulo**, **dirección** y **sentido** (velocidad, fuerza, aceleración).
- En $\mathbb{R}^3$: $\vec{v} = (v_x, v_y, v_z)$.
- **Módulo y vector unitario**

$$\|\vec{v}\| = \sqrt{v_x^2 + v_y^2 + v_z^2}, \quad \hat{v} = \frac{\vec{v}}{\|\vec{v}\|} \quad (\|\hat{v}\| = 1)$$

Descomposición en ejes

$$\vec{v} = v_x \hat{i} + v_y \hat{j} + v_z \hat{k}$$



Operaciones con vectores (básicas)

Suma: $\vec{a} + \vec{b} = (a_x + b_x, a_y + b_y, a_z + b_z)$

Escalar: $\lambda \vec{a} = (\lambda a_x, \lambda a_y, \lambda a_z)$

Distancia: $d(\vec{p}, \vec{q}) = \|\vec{p} - \vec{q}\|$

Proyección de $\vec{a}$ sobre $\vec{b}$: $\text{proj}_{\vec{b}}(\vec{a}) = \left(\frac{\vec{a} \cdot \vec{b}}{\|\vec{b}\|^2} \right) \vec{b}$

Usos en simulación

- Suma/escala: acumulación de fuerzas, interpolaciones (*lerp*).
- Proyección: alinear movimiento con una superficie.

En el ejemplo anterior de PyBullet le pasamos a `resetDebugVisualizerCamera` la posición del objeto como un vector de tres componentes: `cameraTargetPosition=[0, 0, 0]`.

```

1      p.resetDebugVisualizerCamera(
2          cameraDistance=0.173,
3          cameraYaw=180,
4          cameraPitch=-45,
5          cameraTargetPosition=[0,0,0]
6      )
7  
```

Producto escalar

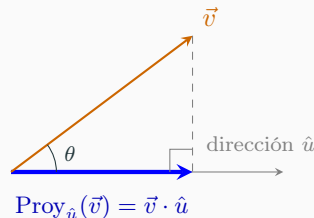
$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y + a_z b_z = \|\vec{a}\| \|\vec{b}\| \cos \theta$$

Propiedades

- Conmuta y es lineal: $\vec{a} \cdot \vec{b} = \vec{b} \cdot \vec{a}$, $\vec{a} \cdot (\lambda \vec{b}) = \lambda(\vec{a} \cdot \vec{b})$.
- Ortogonalidad: $\vec{a} \cdot \vec{b} = 0 \iff \theta = 90^\circ$.

Aplicaciones en simulación

- *Ángulo entre vectores*: comprobación de campo de visión (FOV) o “aim assist”.
- *Proyección de velocidad*: velocidad tangencial = $\vec{v} \cdot \hat{t}$.
- *Potencia*: $P = \vec{F} \cdot \vec{v}$ (útil para depurar fuerzas y movimientos).



Producto vectorial

$$\vec{a} \times \vec{b} = \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix} = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$$

Módulo y dirección

$$\|\vec{a} \times \vec{b}\| = \|\vec{a}\| \|\vec{b}\| \sin \theta \quad (\text{área del paralelogramo}).$$

Dirección dada por la **regla de la mano derecha** (ortogonal a ambos).

Aplicaciones en simulación

- Cálculo de **normales** (iluminación, físicas de contacto).
- **Torque** y momentos angulares en *rigid bodies* 3D.



Mini-ejemplos prácticos

1) Cámara orbital sobre objetivo $\vec{p}_t = (x_t, y_t, z_t)$:

La posición de la cámara $\vec{p}_c$ se obtiene sumando a la posición del objetivo el vector de desplazamiento relativo en esféricas.

$$x_c = x_t + r \sin \theta \cos \varphi$$

$$y_c = y_t + r \sin \theta \sin \varphi$$

$$z_c = z_t + r \cos \theta$$

2) Alineación de movimiento (proyectar $\vec{v}$ sobre “forward” $\hat{f}$):

Descomposición de la velocidad en componentes paralela y perpendicular a la dirección de avance.

$$\vec{v}_{\parallel} = (\vec{v} \cdot \hat{f}) \hat{f}$$

$$\vec{v}_{\perp} = \vec{v} - \vec{v}_{\parallel}$$

3) Normal de una superficie (vía tangentes $\vec{t}_1, \vec{t}_2$):

Cálculo del vector unitario perpendicular al plano definido por dos direcciones tangenciales.

$$\hat{n} = \frac{\vec{t}_1 \times \vec{t}_2}{\|\vec{t}_1 \times \vec{t}_2\|}$$

5 Matrices

Introducción a las Matrices

- Un **matriz** es un objeto matemático utilizado para almacenar conjuntos de datos de manera estructurada.
- Aunque se podrían usar variables separadas, las matrices permiten expresiones matemáticas y código de programas más **compactos y eficientes**. Por ejemplo un sistema de N ecuaciones lineales con N incógnitas se escribe como una sola ecuación matricial.
- En el desarrollo de videojuegos, se utilizan comúnmente para:
 - Modelar el vuelo de aviones.
 - Simular colisiones entre objetos.
 - Resolver sistemas de ecuaciones con múltiples incógnitas.

Sistemas de Ecuaciones en Forma Matricial

Consideremos un sistema de tres ecuaciones con tres incógnitas (x, y, z) :

En forma matricial se expresa como:

$$a = 4x + y - 3z$$

$$b = -7x - 2y + 5z$$

$$c = 3x + 3y + z$$

$$\begin{pmatrix} a \\ b \\ c \end{pmatrix} = A \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 4 & 1 & -3 \\ -7 & -2 & 5 \\ 3 & 3 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

- Esta es una matriz de **dos dimensiones** con tres filas y tres columnas.
- Para resolver el sistema de ecuaciones “basta” con obtener la inversa de la matriz A : A^{-1}
- Las matrices pueden ser de cualquier dimensión (1D, 2D, 3D,...).

Operaciones con Matrices

Suma y Resta

Implica sumar o restar los elementos individuales. Las matrices **deben tener las mismas dimensiones**.

Multiplicación

- Se multiplica cada fila de la primera matriz por cada columna de la segunda.
- **Regla fundamental:** El número de columnas de A debe ser igual al número de filas de B .

Sean las matrices:

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix}, \quad B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \end{pmatrix}$$

El elemento c_{11} del producto $AB = C$ se calcula como:

$$c_{11} = a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31}$$

Matrices de Rotación

Se utilizan para rotar temporalmente los ejes de coordenadas y realizar cálculos, como en colisiones de palos de golf y pelotas.

Si rotamos los ejes x e y un ángulo α sobre el **eje z** :

$$\begin{pmatrix} v'_x \\ v'_y \\ v'_z \end{pmatrix} = \begin{pmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix}$$

- El ángulo θ se conoce también como **ángulo de Euler**.
- Como la rotación es sobre el eje z , la componente v'_z permanece inalterada ($v'_z = v_z$).

Ángulos de Euler

Definición

Representan la orientación de un cuerpo rígido mediante tres rotaciones sucesivas (α, β, γ) respecto a ejes locales o globales.

1. **Precesión** (α): Rotación sobre el eje Z .
2. **Nutación** (β): Rotación sobre el nuevo eje X (eje de nodos).
3. **Rotación propia** (γ): Rotación final sobre el eje Z del cuerpo.

Matriz R

$$R = R_z \cdot R_{x'} \cdot R_{z''}$$

- **Grados de libertad:** Cubren los 3 grados de rotación en $\mathbb{R}^3$.

Matrices de Rotación de Euler

La matriz de rotación total es el producto $R = R_z(\alpha)R_x(\beta)R_z(\gamma)$:

$$R_z(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$R_x(\beta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \beta & -\sin \beta \\ 0 & \sin \beta & \cos \beta \end{pmatrix}$$

$$R_z(\gamma) = \begin{pmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Nota Importante

El orden de multiplicación es crucial. En rotaciones **intrínsecas** (sobre ejes móviles), las matrices se multiplican de izquierda a derecha.

Cálculo de la Trayectoria Final

Ejemplo: Un avión vuela en la dirección del eje Y positivo y vira 20° a estribor y se eleva 30° . Obténgase el nuevo rumbo del avión.

El vector de rumbo inicial será: $\vec{v}_0 = (0, 1, 0)^T$. Aplicamos dos rotaciones sucesivas definidas por los ángulos α y β :

- **Rotación α (sobre eje z):** Giro horizontal ($\alpha = -20^\circ$).
- **Rotación β (sobre eje x'):** Elevación vertical ($\beta = 30^\circ$).

$$\vec{v}_f = R_x(\beta) \cdot R_z(\alpha) \cdot \vec{v}_0$$

$$\vec{v}_f = R_x(\beta) \cdot R_z(\alpha) \cdot \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad \text{con} \quad R_x(\beta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \beta & -\sin \beta \\ 0 & \sin \beta & \cos \beta \end{pmatrix} \quad \text{y} \quad R_z(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Cálculo de la Trayectoria Final

Sustituyendo los valores ($\alpha = -20^\circ$, $\beta = 30^\circ$):

$$\vec{v}_f = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \beta & -\sin \beta \\ 0 & \sin \beta & \cos \beta \end{pmatrix} \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} -\sin \alpha \\ \cos \beta \cos \alpha \\ \sin \beta \cos \alpha \end{pmatrix} \approx \begin{pmatrix} 0,342 \\ 0,814 \\ 0,470 \end{pmatrix}$$

Nota: El tercer ángulo γ representaría la rotación propia sobre el eje de la trayectoria, la cual no altera el vector de dirección final. Por ejemplo, un avión volando de lado como cuando aterrizan en Bilbao.

6 Derivación numérica

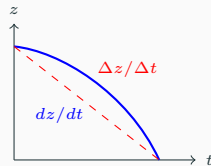
Derivadas en la Física de Videojuegos

- Una **derivada** nos dice cómo cambia una cantidad respecto a otra.
- Proporcionan una **tasa de cambio instantánea**.

Ejemplo: Caída Libre

Si z es la altitud y t el tiempo, la velocidad vertical es:

$$v_z = \frac{dz}{dt}$$



La derivada es la pendiente de la curva en cada punto.

Derivadas de Orden Superior

Es posible tomar la derivada de otra derivada.

- **Primer orden:** Define la velocidad (v) como el cambio de posición respecto al tiempo.
- **Segundo orden:** Define la aceleración (a) como el cambio de velocidad.

Relación Cinemática

La aceleración es la segunda derivada de la posición (z) con respecto al tiempo (t):

$$a_z = \frac{dv_z}{dt} = \frac{d^2z}{dt^2}$$

En un proyectil bajo gravedad constante, la aceleración es horizontal (constante), la velocidad aumenta linealmente y la altitud en función del tiempo es una función cuadrática (parábola).

Derivación Numérica: Diferencias Finitas

En computación, no podemos calcular límites infinitesimales ($dt \rightarrow 0$). En su lugar, usamos un **paso de tiempo discreto** Δt .

Aproximación por Diferencias

Si conocemos el valor de una función en instantes de tiempo separados por un intervalo Δt , la derivada se aproxima como:

$$f'(t) \approx \frac{f(t + \Delta t) - f(t)}{\Delta t}$$

- En videojuegos, Δt suele ser el tiempo entre frames (*delta time*).
- Cuanto más pequeño es Δt , más precisa es la aproximación.
- **Error de truncamiento:** Al ignorar los términos superiores de la serie de Taylor, cometemos un pequeño error numérico.

Esquemas de Diferencias Finitas

Existen tres esquemas básicos:

- **Hacia adelante:** Usa el punto siguiente.

$$f'(t) \approx \frac{f(t + \Delta t) - f(t)}{\Delta t}$$

- **Hacia atrás:** Usa el punto anterior (común en tiempo real).

$$f'(t) \approx \frac{f(t) - f(t - \Delta t)}{\Delta t}$$

- **Centrada:** Usa ambos extremos. Es más precisa.

$$f'(t) \approx \frac{f(t + \Delta t) - f(t - \Delta t)}{2\Delta t}$$

Aplicación en Física

Para calcular la velocidad actual v_i conociendo la posición actual x_i y la del frame anterior x_{i-1} :

$$v_i = \frac{x_i - x_{i-1}}{\Delta t}$$

Implementación: Cálculo de Velocidad

En cada ciclo de la simulación, podemos estimar la velocidad actual comparando la posición nueva con la del frame anterior:

Pseudo-código

```
// Inicialización
x_prev = x_inicial
dt = paso_de_tiempo // (p. ej. 1/60 seg)

BUCLE de simulación:
// 1. Obtener nueva posición del motor físico
x_actual = obtener_posicion()

// 2. Calcular velocidad (Diferencia hacia atrás)
v = (x_actual - x_prev) / dt

// 3. Actualizar estado para el siguiente ciclo
x_prev = x_actual
```

- Este método es fundamental cuando el motor nos da la posición pero no la velocidad instantánea.
- Por ejemplo, Chipmunk2D (pymunk) da la velocidad pero no la aceleración.

7 Integración numérica

El Problema de la Integración

En física y matemáticas, a menudo necesitamos calcular:

$$I = \int_a^b f(x) dx$$

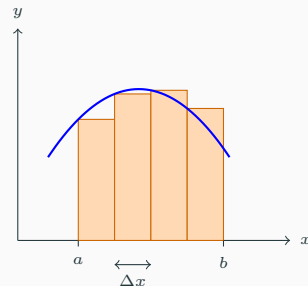
¿Por qué numérica?

- **Sin primitiva conocida:** Funciones como e^{-x^2} (distribución normal) no tienen una solución analítica simple.
- **Datos discretos:** A veces solo tenemos puntos medidos por un sensor (como la posición del péndulo de Pymunk).
- **Complejidad:** La función es tan compleja que es más rápido dejar que el ordenador aproxime el área.

¿En qué consiste el cálculo numérico?

La idea fundamental es la **discretización**:

1. Dividir el intervalo $[a, b]$ en n subintervalos pequeños de ancho Δx .
2. Aproximar el área de cada parte mediante una figura geométrica simple.
3. Sumar todas las áreas.

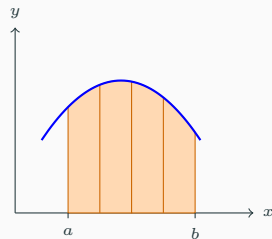
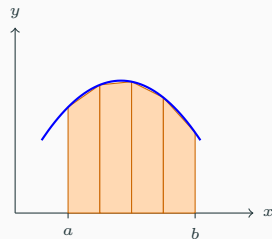


Suma de Riemann (4 intervalos)

- Este método se conoce como **regla del rectángulo**: asume que la función es constante en cada trozo. Error elevado.

Regla de los Trapecios y de Simpson

- Regla de los trapecios
- Aproxima el área bajo la curva usando **trapecios** en lugar de rectángulos.
 - Conecta los puntos $(x_i, f(x_i))$ con segmentos rectos.
 - Reduce el error respecto a los rectángulos.
- Regla de Simpson
 - Utiliza **parábolas** para aproximar la función en cada intervalo.
 - Requiere que los intervalos se agrupen de dos en dos.
 - Es mucho más precisa para funciones suaves.



Comparativa de Métodos

Para una división del intervalo en $\Delta x = (b - a)/n$:

Método	Geometría	Grado Polinomio	Error
Rectángulo	Rectángulo	0 (Constante)	$O(\Delta x)$
Trapezio	Trapezio	1 (Lineal)	$O(\Delta x^2)$
Simpson	Parábola	2 (Cuadrático)	$O(\Delta x^4)$

- El método de **Simpson** es sorprendentemente exacto: incluso integra perfectamente polinomios de grado 3, a pesar de usar parábolas.
- En sistemas físicos, elegir el método adecuado es clave para la **conservación de la energía**: un error acumulado alto puede hacer que los objetos "salgan volando." se detengan sin fricción.
- Cuanto mayor es el orden del error, más grande puede ser el Δt sin sacrificar la estabilidad de la simulación.

8 Resolución numérica de ecuaciones diferenciales

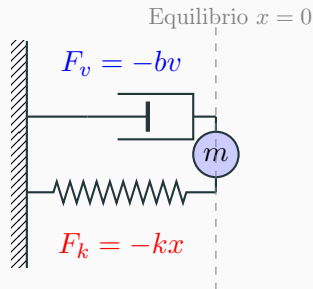
Ecuaciones Diferenciales Ordinarias (ODE)

Una **ODE** es una ecuación donde la incógnita es una función que depende de una sola variable (usualmente el tiempo t).

- En **Dinámica** suelen aparecer ecuaciones donde la incógnita es la posición en función del tiempo y parecen su primera y segunda derivada:
- La **posición** $x(t)$: Situación del objeto en el instante t .
- La **velocidad** $\frac{dx}{dt}$: Primera derivada de la posición.
- La **aceleración** $\frac{d^2x}{dt^2}$: Segunda derivada de la posición.

Resolver la ODE consiste en encontrar la función $x(t)$ que describe el movimiento del objeto.

Ejemplo. Oscilador Amortiguado



- **Fuerza Elástica (F_k):** Recuperadora, depende de la elongación x .
- **Fuerza Viscosa (F_v):** Disipativa, depende de la velocidad v .
- La suma de todas estas fuerzas determina la aceleración en cada momento:

$$ma = \sum F_i = -kx - bv$$

Solución Analítica: Oscilador Amortiguado

Ejemplo: Sistema Masa-Resorte-amortiguador

$$m \frac{d^2 x}{dt^2} - b \frac{dx}{dt} + kx = 0$$

Donde los parámetros definidos son:

- **Masa (m):** 0,2 kg.
- **Amortiguamiento (b):** 0,1 kg/s.
- **Constante (k):** 2,0 N/m.
- **Estado inicial:** Amplitud de 20 cm y reposo ($v_0 = 0$).

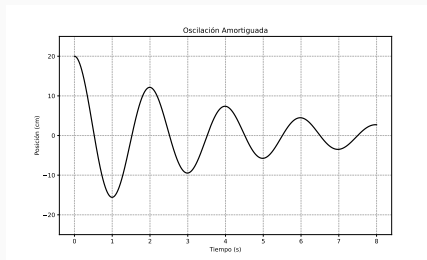


Figura 1: Comportamiento teórico del oscilador.

Esta solución representa el comportamiento físico ideal (exacto) donde la energía se disipa de forma continua.

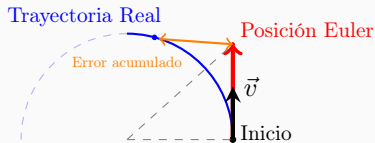
Resolución numérica de ODE: Newton (Euler) vs. Runge-Kutta

Como muchas ODEs no tienen solución analítica sencilla, usamos métodos numéricos para “dar paso” en el tiempo.

Método de Newton (Euler) Es el más simple (aproximación lineal):

$$x_{n+1} = x_n + v_n \Delta t$$

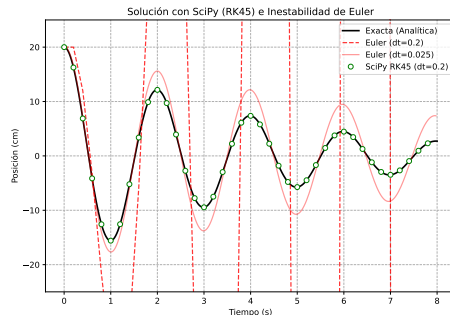
- **Pros:** Muy rápido de calcular.
- **Contras:** Inestable; acumula mucho error en trayectorias con mucha curvatura.



Método de Runge-Kutta (RK4) Evalúa la pendiente en varios puntos (4 pasos) para predecir el siguiente estado.

- **Pros:** Preciso y estable.
- **Contras:** Costoso computacionalmente
- **Runge-Kutta-Fehlberg (RK45)**
Compara soluciones de 4^º y 5^º orden para ajustar automáticamente el paso de tiempo (Δt). Precisión garantizada.

Resolución numérica de ODE: Oscilador Amortiguado



- **Datos:** Masa-resorte con amortiguamiento $b = 0,1 \text{ kg/s}$, $k = 2,0 \text{ N/m}$, $m = 0,2 \text{ kg}$.
- Con $\Delta t = 0,2 \text{ s}$, el método de **Euler** diverge de la trayectoria real.
- Con $\Delta t = 0,025 \text{ s}$ mejora un poco.
- El método **RK45** mantiene la precisión incluso con muestreos gruesos.

Implementación con scipy

```
1  from scipy.integrate import solve_ivp
2  # 1. Definir la funcion del sistema (dy/dt = f(t, y))
3  def sistema(t, y, m, k, b):
4      x, v = y
5      dxdt = v
6      dvdt = (-b*v - k*x) / m
7      return [dxdt, dvdt]
8  # 2. Configurar y lanzar el solver -> y0 es el estado inicial [x0, v0]
9  sol = solve_ivp(sistema,
10                  t_span=(0, 8),          # Intervalo de tiempo (inicio, fin)
11                  y0=[0.2, 0.0],          # Condiciones iniciales
12                  method='RK45',          # El metodo adaptativo
13                  args=(0.2, 2.0, 0.1),    # Parametros m, k, b
14                  rtol=1e-6)               # Tolerancia de error (paso adaptativo)
15  # 3. Acceder a los resultados
16  # sol.t contiene los tiempos que el algoritmo eligio
17  # sol.y[0] contiene las posiciones calculadas
```

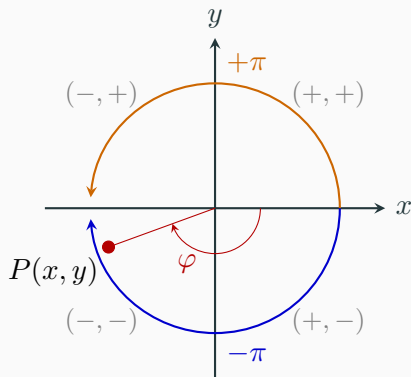
La función $\text{atan2}(y, x)$

La función **arcotangente de dos parámetros** es crucial en robótica y simulación porque:

- Evita la división por cero (cuando $x = 0$).
- Identifica el **cuadrante correcto** observando los signos de x e y .
- Devuelve un rango completo de $(-\pi, \pi]$ (o $[-180^\circ, 180^\circ]$).

Diferencia clave

- $\frac{-1}{-1}$ y $\frac{1}{1}$ dan 1. $\arctan(1)$ siempre devuelve 45° , pero $\text{atan2}(-1, -1)$ devuelve -135° .



Volver

Yaw (ψ) vs. Acimut (φ)

Aunque representan la misma rotación en el plano XY desde el eje $+X$, su interpretación numérica varía según el contexto:

Convención Física (φ)

- **Rango:** $[0, 2\pi)$.
- **Continuidad:** Salta de 2π a 0 al completar una vuelta.

Convención Robótica/Yaw (ψ)

- **Rango:** $(-\pi, \pi]$.
- **Continuidad:** Salta de π a $-\pi$ (el “corte” está atrás).

Implicaciones con `atan2(y, x)`

La función `atan2` devuelve por defecto el rango del **Yaw** $(-\pi, \pi]$.

- Si necesitas $\varphi \in [0, 2\pi)$ para una fórmula física: `if phi < 0: phi += 2*PI`
- **Control:** Es más fácil interpolar un Yaw (ej. de -170° a 170° son solo 20° de giro) que un ángulo acimutal que cruza el cero.

Volver

Dinámica y movimiento para videojuegos y gamificación

Tema 2. Dinámica de la Partícula

Pedro Ángel Bernaola Galván

Febrero 2026

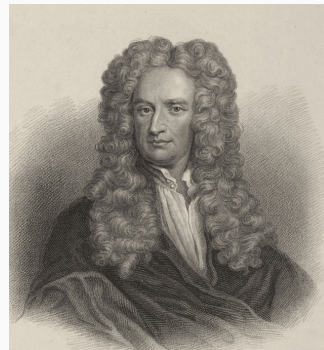
Contenido

- 1 Introducción
- 2 Las leyes de Newton
- 3 Getting started con pymunk y pybullet
- 4 Fuerzas gravitatorias
- 5 Fuerza elástica
- 6 Rozamiento
- 7 Trabajo y energía

1 Introducción

Introducción

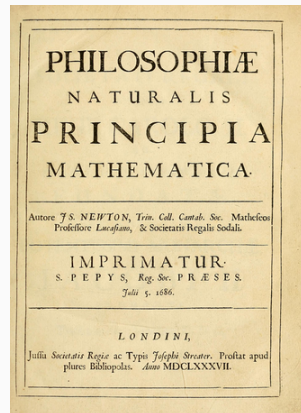
- El capítulo analiza las leyes que relacionan la fuerza, la masa y el movimiento.
- En primer lugar veremos las **Leyes de Newton**.
- Estas leyes son la base de la mecánica clásica y fundamentales para cualquier motor de física.
- **Concepto clave:** La fuerza es una cantidad vectorial (tiene magnitud y dirección).
- Veremos fuerzas que aparecen con frecuencia en las simulaciones: **gravitatoria, de rozamiento y fuerzas elásticas**.



2 Las leyes de Newton

Leyes de Newton

1. **Primera Ley:** Ley de la Inercia.
 2. **Segunda Ley:** Ley Fundamental de la Dinámica
($F = ma$).
 3. **Tercera Ley:** Ley de Acción y Reacción.
- **Publicación:** 5 de julio de 1687.
 - **Obra:** *Philosophiæ Naturalis Principia Mathematica*.
 - **Impacto:** Establecieron las bases de la mecánica clásica que usamos hoy en simulaciones.



Primera Ley de Newton: Inercia

- Un objeto en reposo permanece en reposo y un objeto en movimiento permanece en movimiento a velocidad constante a menos que actúe sobre él una fuerza externa neta.
- La **masa** es la medida de la inercia de un objeto (lo fácil o difícil que es cambiar su velocidad).



$$\text{Si } \sum \vec{F} = 0 \implies \vec{a} = 0 \implies \vec{v} = cte$$



Primera Ley de Newton: Inercia

- Debido a la inercia, en el espacio es tan difícil acelerar un objeto como detenerlo: ambos requieren una fuerza externa.
- Las naves regresan a velocidades orbitales extremas (aprox. **28,000 km/h**).
- No disponen de combustible suficiente para frenar usando motores; requerirían un cohete tan grande como el de salida.
- La solución es utilizar la **fricción atmosférica** como "freno natural", transformando la energía cinética en calor intenso.



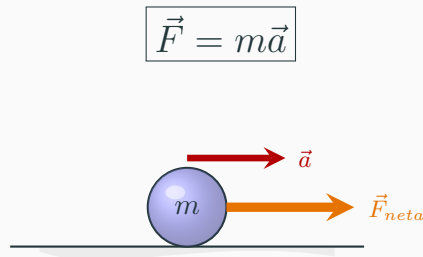
La atmósfera como freno.

Segunda Ley de Newton

- La **Segunda Ley** define la fuerza como la causa del cambio de movimiento.
- El efecto de una fuerza sobre la velocidad es inversamente proporcional a la masa (inercia).

$$\vec{a} = \frac{\sum \vec{F}_{neta}}{m}$$

- Se considera la definición operacional de la **Fuerza**.
- Sus unidades $[F] = \text{kg} \frac{\text{m}}{\text{s}^2} \equiv \text{N}$ (newton)



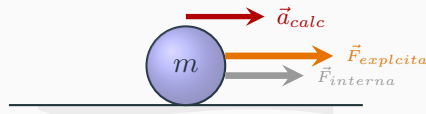
Segunda Ley: La Realidad del Motor Físico

- **Caja Negra:** Algunos motores (Pymunk/PyBullet) ocultan las *fuerzas internas* (fricción, normal, impulsos de colisión).
- No solemos tener acceso a la *fuerza total* (el motor oculta fricciones y colisiones internas).
- El programador solo inyecta **fuerzas explícitas**.
- La aceleración no es un dato, es una consecuencia numérica:

Cálculo de la aceleración:

$$\vec{a}_{calc} \approx \frac{\vec{v}_n - \vec{v}_{n-1}}{\Delta t}$$

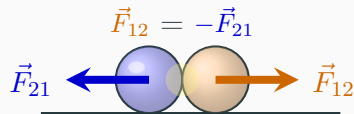
Derivada numérica utilizada en la simulación.



Tercera Ley de Newton: Acción y Reacción

- Siempre que un objeto ejerce una fuerza sobre otro, el segundo ejerce una fuerza de igual magnitud y dirección opuesta.
- **Propulsión a chorro:** El cohete acelera hacia adelante al empujar los gases de escape hacia atrás.
- **Importante:** No se empuja “contra el suelo”; la interacción es entre el motor y el gas. Por eso los cohetes funcionan en el vacío del espacio.
- *"Si el cohete empuja al gas, el gas empuja al cohete".*

$$\vec{F}_{\text{cohete}} = -\vec{F}_{\text{gases}}$$



Acción y reacción en el despegue.

3 Getting started con pymunk y pybullet

Tutorial Pymunk (1): Configuración de la ventana

Configuración del entorno:

- `import`: Cargamos `pymunk` para el motor de física y `pygame` para la renderización gráfica.
- `pygame.init()`: Prepara el hardware y los módulos (sonido, video, eventos) para trabajar.
- `set_mode`: Define el lienzo de dibujo. En este caso, un cuadrado de 600×600 píxeles.

Código Python

```
1 import pymunk
2 import pygame
3 #1. Configuración de la ventana
4 pygame.init()
5 pantalla = pygame.display.set_mode((600, 600))
6 pygame.display.set_caption("Hello Munk")
7 reloj = pygame.time.Clock()
8
```

- `set_caption`: Establece el nombre de la aplicación en la barra de tareas.
- `Clock()`: Elemento crítico para sincronizar los pasos físicos con los frames visuales.

Tutorial Pymunk (2): Mundo, Suelo y Cuerpo Rígido

El motor físico:

- **Space:** Es el “Mundo”.
- **Suelo (Segment):** Un cuerpo estático (static_body). No es obligatorio.
- **Caja (Body):**
 - **Masa y Momento:** Necesarios para que el motor sepa cómo se desplaza y cómo rota.
 - **Posición:** Se sitúa en (300, 50) (parte superior central).
- **add:** Es vital añadir tanto el Body (física) como la Shape (forma) al espacio.

Código: Mundo y Objetos

```

1  # 2. Configuración del mundo físico
2  espacio = pymunk.Space()
3  # --- CREAR EL SUELO ---
4  suelo = pymunk.Segment(espacio.static_body,
5  (0, 550), (600, 550), 5)
6  suelo.elasticity = 0.5
7  espacio.add(suelo)
8  # --- CREAR LA CAJA ---
9  masa = 1
10 lado = 50
11 momento = pymunk.moment_for_box(masa, (lado, lado))
12 cuerpo_caja = pymunk.Body(masa, momento)
13 cuerpo_caja.position = (300, 50)
14 forma_caja = pymunk.Poly.create_box(cuerpo_caja, (
15     lado, lado))
16 forma_caja.elasticity = 0.8
17 espacio.add(cuerpo_caja, forma_caja)

```

¡OJO!: distingue entre el cuerpo (body) y la forma (shape)

Tutorial Pymunk (3): Bucle y Renderizado

Actualización y Dibujo:

- `while running`: Mantiene el programa activo hasta cerrar la ventana.
- `espacio.step()`: Avanza la simulación física. Se recomienda un paso fijo (1/60) para mayor estabilidad.
- **Renderizado:**
 - `fill`: Limpia la pantalla cada frame.
 - `pygame.draw.rect`: Dibuja la caja usando la posición del cuerpo de Pymunk.
- `clock.tick(60)`: Sincroniza el bucle a 60 cuadros por segundo.

```
1 # 3. Bucle principal
2 running = True
3 while running:
4     for event in pygame.event.get():
5         if event.type == pygame.QUIT:
6             running = False
7     pantalla.fill((255, 255, 255))
8     # Avanzar fisica
9     espacio.step(1/60.0)
10    # Dibujar caja (posicion actual)
11    pos = cuerpo_caja.position
12    pygame.draw.rect(pantalla, (255, 0, 0),
13                     (pos.x-25, pos.y-25, 50, 50))
14    # Dibujar suelo
15    pygame.draw.line(pantalla, (0, 0, 0),
16                     (0, 550), (600, 550), 5)
17    pygame.display.flip()
18    reloj.tick(60)
19 pygame.quit()
20
```

Tutorial Pymunk (4): Métodos de Dibujo

1. Dibujo Manual (Pygame): Existen funciones específicas según la forma física:

- `draw.circle()`: Para círculos.
- `draw.line()`: Para segmentos/suelos.
- `draw.polygon()`: Para cajas o formas irregulares.

Nota: Se calculan rotaciones y centros manualmente.

2. Dibujo Automático (Debug): Pymunk ofrece `DrawOptions`, que sincroniza automáticamente la posición y la **rotación** de todos los objetos en el espacio.

Aquí `pantalla` es el objeto tipo `surface` que creamos al principio al llamar a

`pygame.display.set_mode((600, 600))`

Uso de Debug Draw

```

1  import pymunk.pygame_util
2
3  # 1. Configurar opciones (fuera del
   bucle)
4  opciones = pymunk.pygame_util.
   DrawOptions(pantalla)
5
6  # 2. En el bucle principal
7  while running:
8      # ... eventos y step ...
9
10     pantalla.fill((255, 255, 255))
11
12     # Dibuja todo el espacio
   automaticamente
13     espacio.debug_draw(opciones)
14
15     pygame.display.flip()
16

```

Tutorial Pybullet (1): Mundo y Suelo 3D

Entorno 3D:

- `p.GUI`: Inicia la simulación con una ventana gráfica 3D interactiva.
- `getDataPath()`: Configura la ruta para encontrar modelos estándar (como el suelo).
- `loadURDF`: Carga objetos desde archivos. El *plane.urdf* es el suelo infinito.
- `restitution`: Es el término técnico para el “rebote”. El valor `-1` indica que se aplica a la base del objeto.

Código: Inicialización Pybullet

```
1  import pybullet as p
2  import pybullet_data
3
4  # 1. Configuración inicial
5  p.connect(p.GUI)
6  p.setAdditionalSearchPath(
7      pybullet_data.getDataPath())
8
9  # 2. Cargar suelo y configurar rebote
10 suelo_id = p.loadURDF("plane.urdf")
11 # restitution = elasticidad
12 p.changeDynamics(suelo_id, -1,
13     restitution=0.7)
```

Tutorial Pybullet (2): Objeto y Cámara

Cuerpos y Visualización:

- `loadURDF`: Cargamos el cubo pequeño. El segundo argumento `[0, 0, 1.0]` establece su posición inicial (X, Y, Z) . Al estar en $Z = 1$, caerá un metro.
- `restitution`: Al igual que con el suelo, asignamos un rebote del 70%. Si ambos tienen restitución, el efecto se combina: **es el producto de las dos.**

Código: Cubo y Cámara

```

1  # 3. Cargar cubo y configurar su rebote
2  # Posicion Z=1.0 (1 metro de altura)
3  cubo_id = p.loadURDF("cube_small.urdf", [0, 0, 1.0])
4  p.changeDynamics(cubo_id, -1, restitution=0.7)
5
6  # 4. Ajustar camara
7  p.resetDebugVisualizerCamera(
8      cameraDistance=1.5,
9      cameraYaw=45,
10     cameraPitch=-30,
11     cameraTargetPosition=[0, 0, 0])
12

```

- **Cámara:**
 - `cameraDistance`
 - `Yaw/Pitch`
 - `TargetPosition`

Tutorial Pybullet (3): Bucle de Simulación

Ejecución de la Física:

- `stepSimulation`: Ordena al motor calcular el siguiente estado (posiciones, colisiones, velocidades).
- `time.sleep`: Pybullet corre a 240Hz por defecto. Pausamos 1/240 segundos para que la simulación coincida con el tiempo real.
- **Cálculo del Rebote:**
 - La restitución final se calcula como el producto de ambas:

$$e_{total} = e_{suelo} \times e_{cubo}.$$
 - En este ejemplo: $0,7 \times 0,7 = 0,49$.

Código: El Bucle Infinito

```

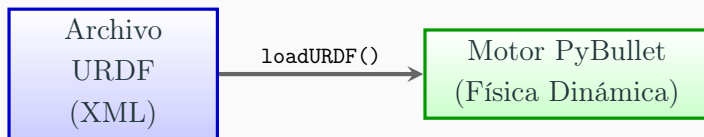
1  import time
2  # 5. Bucle de simulacion
3  print("Simulacion iniciada...")
4  try:
5      while True:
6          # Avanza el motor fisico
7          p.stepSimulation()
8          # Sincroniza con tiempo real (240Hz)
9          time.sleep(1./240.)
10 except KeyboardInterrupt:
11     # Cierra la conexion al salir
12     p.disconnect()
13     print("\nConexion cerrada.")
14 
```

- **Control de salida:** El bloque `try/except` asegura que la conexión se cierre limpiamente al interrumpir.

Tutorial Pybullet (4): Archivos .urdf

Al cargar un archivo .urdf en PyBullet mediante `loadURDF()`, el motor no solo importa la geometría visual, sino que extrae propiedades físicas críticas definidas en el XML:

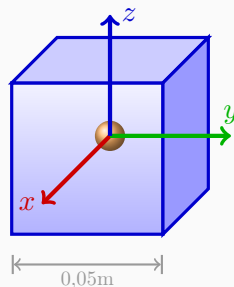
- **Masa e Inercia:** Definidas en la etiqueta `<inertial>`, fundamentales para la simulación de la dinámica.
- **Fricción y Restitución:** Parámetros de contacto especificados en `<contact>`.
- **Límites de Articulación:** Esfuerzo máximo, velocidad y límites de posición en los `<joint>`.



Tutorial Pybullet (5): Propiedades de `cube_small.urdf`

El archivo `cube_small.urdf` es un estándar en PyBullet para tareas de manipulación. Sus propiedades físicas están optimizadas para la estabilidad en el contacto:

- **Dimensiones:** Cubo de $5 \times 5 \times 5$ cm (0,05 m).
- **Masa e Inercia:** 0,05 kg; $I_{i,i} \approx 2,08 \times 10^{-5}$ kg/m².
- **Fricción:** Coeficientes elevados para evitar deslizamientos en el agarre.
- **Extracción Dinámica:** Usamos `getDynamicsInfo()` para obtener masa, fricción e inercia del link.
- **Datos Visuales:** `getVisualShapeData()` permite recuperar las dimensiones y escala de la malla.



Tutorial Pybullet (6): Fichero cube_small.urdf

Si los archivos no han sido creados o descargados manualmente, PyBullet utiliza sus propios archivos estándar ubicados en el paquete de datos:

Ruta del sistema:

```
<entorno>/lib/python3.10/
site-packages/pybullet_data
```

- Contiene geometrías (.obj, .stl).
- Incluye robots (Panda, Kuka).
- **Carga:** Se accede mediante la librería `pybullet_data`.

```

1  <?xml version="0.0" ?>
2  <robot name="cube.urdf">
3    <link name="baseLink">
4      <contact>
5        <lateral_friction value="1.0"/>
6      </contact>
7      <inertial>
8        <mass value=".1"/>
9        <inertia ixx="1" ixy="0" ixz="0"
10         iyy="1" iyz="0" izz="1"/>
11      </inertial>
12      <visual>
13        <geometry>
14          <mesh filename="cube.obj"
15            scale=".05 .05 .05"/>
16        </geometry>
17      </visual>
18      <collision>
19        <geometry>
20          <box size=".05 .05 .05"/>
21        </geometry>
22      </collision>
23    </link>
24  </robot>
25
```

Renderizado: Pymunk vs. Pybullet

Gestión Visual:

- **Pymunk:** Es un motor “ciego”. Solo calcula números. Necesitas una librería externa (**Pygame**) y programar cada dibujo manualmente.
- **Pybullet (GUI):** Es un entorno integral. Al conectar con `p.GUI`, el motor de renderizado OpenGL se encarga de todo.

El Modo `p.DIRECT`:

- Ejecuta la física **sin ventana gráfica**.
- **Uso principal:** Entrenamiento de Redes Neuronales y Reinforcement Learning.
- **Ventaja:** Es mucho más rápido al no gastar recursos en procesar imágenes.

Flujo de Trabajo

```
1  # EN PYMUNK + PYGAME
2  # Tienes que "pintar" cada frame
3  pygame.draw.rect(pantalla, rojo,
4                   pos_caja)
5  # EN PYBULLET (GUI)
6  # El renderizado es automatico
7  p.stepSimulation()
8  # EN PYBULLET (DIRECT)
9  # Solo calculos, ideal para
10 servidores
11 p.connect(p.DIRECT)
12 for _ in range(10000):
13     p.stepSimulation()
```

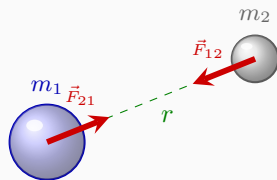
4 Fuerzas gravitatorias

Ley de Gravitación Universal

- Existe una fuerza de atracción entre cualquier par de objetos con masa.
- Esta fuerza es:
 - Proporcional al producto de sus masas.
 - Inversamente proporcional al cuadrado de la distancia (r).

$$F_G = G \frac{m_1 m_2}{r^2}$$

- $G = 6,674 \times 10^{-11} \text{ N} \cdot \text{m}^2/\text{kg}^2$
- **Aplicación:** Crucial para calcular trayectorias de cohetes y órbitas satelitales (como en la misión Artemis).



Trayectorias de la misión Artemis.

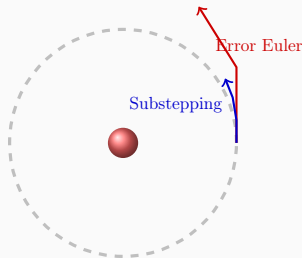
gravitacion03.py

Estabilidad Numérica: Substepping

Las fuerzas centrales son extremadamente sensibles al paso de tiempo (dt).

El *substepping* mejora la fidelidad (véase `gravitacion01.py` y `gravitacion02.py`):

- **Error de Discretización:** En pasos grandes, el objeto “escapa” de la curva real al moverse en líneas rectas demasiado largas.
- **División del Tiempo:** Dividir el `step` de un frame en N sub-pasos reduce el error de energía acumulado.
- **Implementación:**
 - Se itera N veces el cálculo de fuerza.
 - Se ejecuta `space.step(dt/N)`.
- **Resultado:** Órbitas estables sin necesidad de integradores complejos.

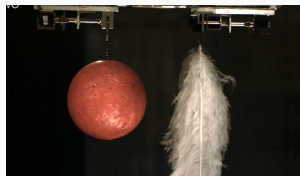
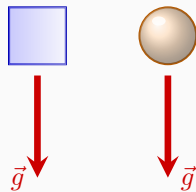


Gravedad en la Tierra (g)

- La aceleración de la gravedad es independiente de la masa del objeto que cae.
- En la superficie terrestre, simplificamos la Ley Universal como:

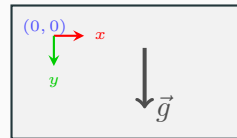
$$P = F_g = mg$$

- Donde $g \approx 9,81 \text{ m/s}^2$.
- **Concepto clave:** En ausencia de aire (vacío), una bola y una pluma caen exactamente a la misma velocidad.
- En otros cuerpos celestes el valor cambia (p.ej. Luna: $1,62 \text{ m/s}^2$).



Implementación: Pymunk (2D)

- **Espacio de trabajo:** Se define a nivel global en el objeto `Space`.
- **Unidades:** ¡Cuidado! Pymunk no usa metros, usa **píxeles**.
- **Escala:** Para una gravedad natural, se usan valores entre 600 y 1000 px/s^2 .
- Las velocidades resultantes se medirán en px/s .



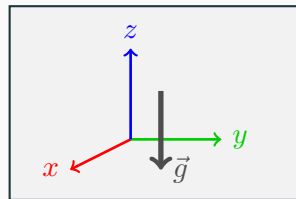
Y aumenta hacia abajo

Código Python

```
1 import pymunk
2 space = pymunk.Space()
3 # g en px/s^2 (vector x, y)
4 space.gravity = (0, 900)
5
```

Implementación: PyBullet (3D)

- **Simulación 3D:** La gravedad requiere un vector de tres dimensiones (x, y, z).
- **Eje vertical:** Por defecto, la gravedad actúa sobre el eje **Z**.
- **Unidades:** PyBullet está orientado a la robótica, por lo que usa unidades del SI (**metros/s²**).
- Objetos con `mass=0` ignoran la gravedad (son estáticos).
- Convenio de colores de los ejes
(X, Y, Z) $\rightarrow$ (R, G, B)



Código Python

```
1 import pybullet as p
2 p.connect(p.GUI)
3 # g en m/s^2 (vector x, y, z)
4 p.setGravity(0, 0, -9.81)
5
```


Comparativa: Pymunk vs. PyBullet

Característica	Pymunk (2D)	PyBullet (3D)
Unidades	Píxeles	Metros
Gravedad típica	(0, 900)	(0, 0, -9,81)
Visualización	Externa (Pygame)	Nativa (p.GUI)
Uso principal	Juegos 2D, teoría	Robótica, juegos 3D

Consejo para desarrolladores

Si vuestro proyecto es en 2D, **Pymunk** es mucho más ligero y fácil de integrar. Para simulaciones físicas realistas o robótica, **PyBullet** es el estándar de la industria.

Ejemplo. Tiro Parabólico

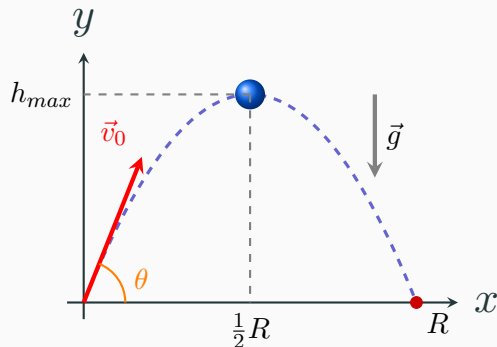
El movimiento de un proyectil bajo la aceleración constante de la gravedad g se descompone en dos ejes independientes:

Eje Horizontal (MRU):

- $v_x = v_0 \cos \theta$
- $x = x_0 + v_x t$

Eje Vertical (MRUA):

- $v_y = \underbrace{v_0 \sin \theta}_{v_{0y}} - gt$
- $y = y_0 + v_{0y}t - \frac{1}{2}gt^2$



Ejemplos: `disparo02.py` (Pymunk) `disparo3D_02.py` (PyBullet)

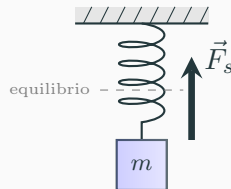
5 Fuerza elástica

Fuerza Elástica: Ley de Hooke

- Modela la fuerza ejercida por muelles o materiales elásticos.
- La fuerza (*aproximadamente*) es proporcional al desplazamiento (x) desde la posición de equilibrio:

$$F_s = -kx$$

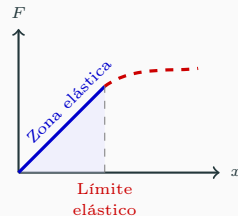
- Donde:
 - k : Constante elástica (rigidez) $[k] = \text{N/m}$.
 - El signo negativo indica que es una **fuerza restauradora**.
- **Aplicación:** Fundamental para simular suspensiones de vehículos, cuerdas elásticas y algunas colisiones.
- Lo veremos en detalle luego cuando veamos el rozamiento.



Más allá de la Ley de Hooke

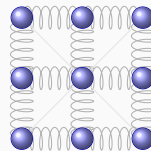
- **Límite Elástico:**

- La ley de Hooke es solo una aproximación para desplazamientos pequeños.
- Si estiramos demasiado el material (x es muy grande), la ley deja de ser lineal.
- El objeto puede sufrir deformaciones permanentes o romperse.



- **Modelado de "Soft Bodies":**

- Uniendo varias masas con muelles (k) podemos simular:
 - Ropa, banderas y pelo dinámico.
 - Gelatinas o cuerpos blandos.



Malla de Muelles (Física de Telas)

Dinámica del Péndulo Simple

La dinámica del péndulo está gobernada por la componente tangencial del peso:

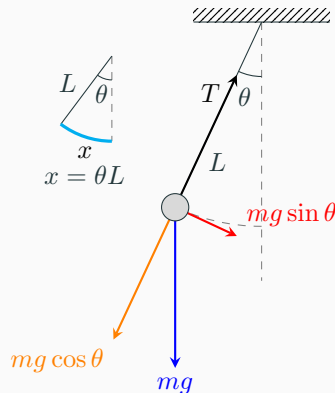
- **Fuerza recuperadora:**

$$F = -mg \sin \theta$$

Apunta siempre hacia el equilibrio.

- **Variable de arco (x):** La distancia sobre la trayectoria es $x = \theta L$.
- **Aproximación lineal:** Para ángulos pequeños ($\sin \theta \simeq \theta$):

$$F \simeq -mg\theta = -mg\frac{x}{L}$$

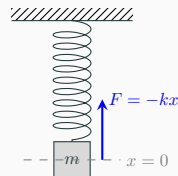


Analogía: El Péndulo vs. El Muelle

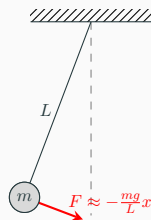
Se comporta como un muelle en el que la fuerza “elástica” es la gravedad (componente tangencial de la fuerza)

Propiedad	Muelle	Péndulo
Fuerza Recuperadora	$F = -kx$	$F \simeq -\frac{mg}{L}x$
Frecuencia Angular (ω)	$\sqrt{\frac{k}{m}}$	$\sqrt{\frac{g}{L}}$
Variable de estado	Elongación (x)	Arco ($x = L\theta$)

- Ambos sistemas son equivalentes cuando la fuerza recuperadora es linealmente proporcional al desplazamiento. Esto ocurre en el péndulo para ángulos pequeños ($\sin \theta \simeq \theta$).



Muelle (OAS)



Péndulo ($\theta \ll 1$)

6 Rozamiento

Rozamiento. Conceptos fundamentales

El **rozamiento** es cualquier fuerza que se opone al movimiento y transforma la energía mecánica en calor

- **Fricción (Sólida/Coulomb):**

- Se origina en el contacto entre superficies rugosas.
- Magnitud: $F_r \leq \mu N$ ($N \rightarrow$ fuerza normal a la superficie de contacto).
- Es independiente de la velocidad.

- **Damping (Viscoso/Amortiguamiento):**

- Se origina por la resistencia de un fluido o histéresis interna.
- Magnitud: Depende de la velocidad: $F_v = -f(v)$. Siempre opuesta a $\vec{v}$
- Se suele tomar $F_v = -bv$, aunque puede aparecer $F_v \propto v^\alpha$ con $\alpha > 1$

Nomenclatura

Usamos **Fricción** para el contacto suelo-objeto y **Damping** (rozamiento viscoso en castellano) para la resistencia del aire o el comportamiento interno de muelles y cuerdas.

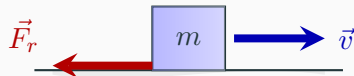
Fricción

- La fricción es la fuerza que se opone al movimiento relativo entre materiales sólidos.
- Depende de la naturaleza de las superficies (coeficiente μ) y de la fuerza normal (N)
- **Fricción Estática** (μ_e): Resistencia para iniciar el movimiento.

$$F_r \leq \mu_e N$$

- **Fricción Dinámica** (μ_d): Resistencia durante el deslizamiento ($\mu_d < \mu_e$).

$$F_r = \mu_d N$$



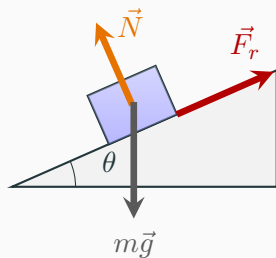
Ejemplos

`friccion01.py`

`friccion02.py`

Fricción en Planos Inclinados

- En una rampa, la fuerza normal se reduce: $N = mg \cos(\theta)$.
- Por lo tanto, la fuerza de fricción también disminuye a medida que aumenta la pendiente.
- **En videojuegos:** Fundamental para decidir si un objeto se queda estático o empieza a resbalar por una pendiente.



`friccion03.py`

Damping o rozamiento viscoso: el modelo lineal

En medios fluidos o sistemas mecánicos lubricados, la resistencia al movimiento se modela mediante fuerzas disipativas que dependen de la velocidad.

- **Definición:** La fuerza de rozamiento viscoso es directamente proporcional a la velocidad del objeto $\vec{F}_v = -b \cdot \vec{v}$

Donde b es el coeficiente de amortiguamiento o **damping**. $[b] = \text{N}\cdot\text{s}/\text{m} = \text{kg}/\text{s}$

- **Sentido:** El signo negativo indica que la fuerza siempre se opone al vector velocidad, extrayendo energía del sistema.

Analogía Física

Este modelo es equivalente al de un amortiguador hidráulico o al de una esfera pequeña moviéndose lentamente en un fluido denso (Régimen de Stokes).

Implementación en Pymunk: `space.damping`

Pymunk simplifica el cálculo de la fricción viscosa a través de un parámetro global en el objeto `Space`.

- **Mecanismo:** En lugar de calcular una fuerza en Newtons, Pymunk escala la velocidad en cada paso de tiempo (*step*).
- **Algoritmo de actualización:** $v_{\text{new}} = v_{\text{old}} * \text{damping} * dt$
- **Valores del parámetro:**
 - `damping = 1.0`: No hay pérdida de energía (vacío perfecto).
 - `damping = 0.9`: El cuerpo pierde un 10 % de su velocidad por unidad de tiempo.
- **Ventaja:** Muy eficiente, no hay que calcular fuerzas para cada objeto.
- **Inconvenientes:**
 - Es igual para todos, no se puede tener en cuenta datos reales de los cuerpos (e.g. aerodinámica, textura, forma)
 - No es directo relacionar el valor de `damping` con la constante de amortiguamiento viscoso (b)

Damping en Motores 3D: PyBullet

Al igual que Pymunk, PyBullet utiliza el modelo de **fricción viscosa lineal** para estabilizar la simulación y simular medios fluidos.

- **Desacoplamiento:** A diferencia de los motores 2D simples, permite separar el `linearDamping` del `angularDamping`.
- **Ecuación de integración:** PyBullet integra estas fuerzas de amortiguamiento dentro del *solver* de restricciones:

$$v_{t+1} = v_t - \frac{b}{m} \cdot v_t \cdot dt$$

- **Granularidad:** Se define por cada objeto (*body*) individualmente, no de manera global para todo el espacio.
- Además de simular aire, el damping en PyBullet es crítico para evitar que los robots u objetos “vibren” o ganen energía infinita debido a errores numéricos del integrador.

Muelle Amortiguado

Ecuación Diferencial

El movimiento de una masa m sujeta a un muelle con amortiguamiento lineal sigue:

$$m \frac{d^2x}{dt^2} + b \frac{dx}{dt} + kx = 0$$

Constantes Físicas:

- k : Constante elástica o rigidez.
- b : Coeficiente de amortiguamiento.
- En simulaciones, si $b^2 \simeq 4mk$, el sistema alcanza el **amortiguamiento crítico**.
- **Tiempo característico**: $\tau = \frac{1}{\gamma} = \frac{2m}{b}$

Unidades (SI vs Pymunk)

Parámetro	S.I.	Pymunk
Masa (m)	kg	unidad masa
Distancia (x)	m	píxeles
Rigidez (k)	N/m	$\frac{\text{masa}}{\text{tiempo}^2}$
Amort. (b)	N·s/m = kg/s	$\frac{\text{masa}}{\text{tiempo}}$

Nota: Pymunk es adimensional por defecto. Si usas píxeles como metros y segundos reales, las constantes escalan proporcionalmente a la masa definida.

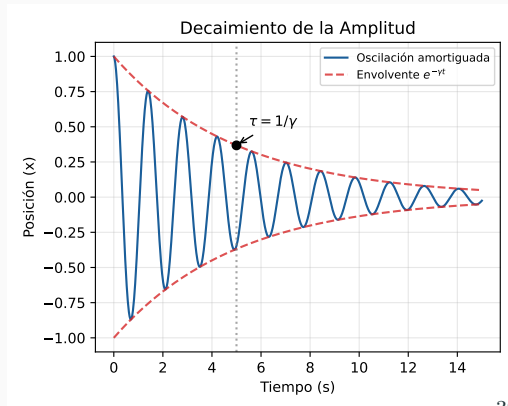
Tiempo de característico (τ)

El tiempo característico o tiempo de relajación, τ , representa la escala temporal en la que la energía del sistema se disipa de forma significativa.

Interpretación Física

$$\tau = \frac{1}{\gamma} = \frac{2m}{b}$$

- Es el tiempo necesario para que la amplitud caiga a $1/e$ ($\simeq 37\%$) de su valor inicial.
- Cuanto mayor es el amortiguamiento b , menor es τ y más rápido se detiene el sistema.
- Define la "memoria" del sistema respecto a su estado inicial.

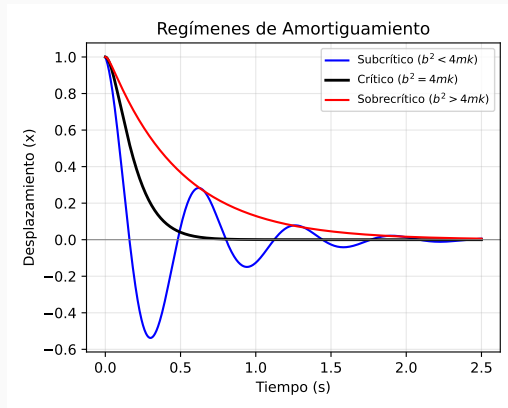


Regímenes de Amortiguamiento

La respuesta del sistema depende de la relación entre la disipación (b) y la restauración (k).

Clasificación Física

- **Subcrítico** ($b^2 < 4mk$): El sistema oscila con amplitud decreciente.
- **Crítico** ($b^2 = 4mk$): El sistema regresa al equilibrio lo más rápido posible sin oscilar.
- **Supercrítico** ($b^2 > 4mk$): Fricción excesiva. El regreso al equilibrio es lento y "viscoso".



Muelles Amortiguados: DampedSpring en Pymunk

Parámetros:

- **a y b**: Cuerpos unidos a los extremos del muelle (estáticos o dinámicos)
- **anchor_a y anchor_b**: Puntos de anclaje
- **rest_length**: Longitud del muelle en reposo.
- **stiffness**: La k del muelle (recordar las unidades “peculiares” de Pymunk)
- **damping**: Coeficiente de amortiguamiento b , el que aparece en la ecuación diferencial, no un porcentaje como en el damping espacial

muelle01.py

muelle_doble01.py

Código Python

```

1  # Creamos el muelle amortiguado
2  muelle = pymunk.DampedSpring(
3      a=cuerpo_a,
4      b=cuerpo_b,
5      anchor_a=punto_anclaje_a,
6      anchor_b=punto_anclaje_b,
7      rest_length=longitud_reposo,
8      stiffness=rigidez, # la k del
9                          muelle
10     damping=amortiguamiento # la b
11 )
12 # IMPORTANTE incluirlo en el
13 espacio
    espacio.add(muelle)

```

7 Trabajo y energía

El Concepto de Trabajo (W)

El trabajo es el resultado de la aplicación de una fuerza a lo largo de una distancia

- El trabajo se define formalmente como el **producto escalar** del vector fuerza ($\vec{F}$) por el vector desplazamiento ($\vec{d}$):

$$W = \vec{F} \cdot \vec{d} = Fd \cos \theta$$

- Donde θ es el ángulo entre la fuerza y la dirección del movimiento.
- **Interpretación:** Solo la componente de la fuerza que actúa en la dirección del movimiento realiza trabajo.
- **Unidades:** Se mide en **julios (J)**, donde $1 \text{ J} = 1 \text{ N} \cdot \text{m}$.

Energía: Cinética y Potencial

La energía es la capacidad de un sistema para realizar trabajo.

Energía Cinética (E_c)

Energía debida al movimiento. Fundamental para calcular impactos y colisiones en el motor físico.

$$E_c = \frac{1}{2}mv^2$$

Energía Potencial Gravitatoria (E_p)

Depende de la configuración del sistema bajo la fuerza de gravedad.

- **Aproximación Local:** Si **no** estamos en un problema orbital o de distancias astronómicas, usamos:

$$E_p = mgh$$

- Donde $g \approx 9,81 \text{ m/s}^2$ es la aceleración constante cerca de la superficie terrestre.

Energía Potencial Elástica (E_k)

Energía almacenada al comprimir o extender un muelle: $E_k = \frac{1}{2}k\Delta x^2$

Potencia (P)

La potencia es la proporción a la que se transfiere energía o se realiza un trabajo en el tiempo.

$$P = \frac{W}{\Delta t}$$

- Se mide en watios (W), $1 \text{ W} = 1 \text{ J/s}$
- **Visión para Vehículos (Dinámica):** Para aplicaciones como el movimiento de coches o naves, es muy útil expresarla como el **producto escalar de la fuerza por la velocidad**:

$$P = \vec{F} \cdot \vec{v}$$

- **Importancia en Gameplay:**
 - Permite calcular la potencia necesaria para mover un objeto a una velocidad dada conociendo la fuerza que se opone (e.g. rozamiento con el aire)
 - Fundamental para simular el par motor y los límites de aceleración de vehículos.

Dinámica y movimiento para videojuegos y gamificación

Tema 3. Dinámica de rotación. Sólido rígido

Pedro Ángel Bernaola Galván

Febrero 2026

Contenido

- 1 Introducción
- 2 Fuerza centrípeta
- 3 Centro de masas
- 4 El Sólido Rígido. Momento de inercia
- 5 Rodadura y deslizamiento
- 6 Orientación en 3D
- 7 Rozamiento por Rodadura

1 Introducción

Dinámica de Rotación: Introducción

En el desarrollo de videojuegos, no basta con que los objetos se desplacen; deben girar, volcar y reaccionar a colisiones excéntricas.

Pasamos del modelo de partícula al de sólido con extensión espacial:

- **Pymunk:** Maneja rotaciones 2D mediante el momento de inercia y el *torque*.
- **PyBullet:** Simula rotaciones en 3D (cuaterniones y matrices de rotación).
- **Objetivo:** Relacionar las fuerzas aplicadas con el cambio en el estado de rotación.

Cinemática Angular

Para describir el giro, definimos magnitudes análogas a las lineales:

Magnitudes fundamentales

- **Espacio angular (θ):** Ángulo recorrido en radianes.
- **Velocidad angular (ω):** Variación del ángulo por unidad de tiempo.

$$\omega = \frac{d\theta}{dt}$$

- **Aceleración angular (α):** Variación de la velocidad angular.

$$\alpha = \frac{d\omega}{dt} = \frac{d^2\theta}{dt^2}$$

Nota: En Pymunk, la velocidad angular se mide en rad/s.

2 Fuerza centrípeta

La Física del Giro: Fuerza Centrípeta

- **Definición:** Es la componente de la fuerza neta que actúa de forma perpendicular a la velocidad instantánea del cuerpo en cada punto del camino.
- Si queremos que un cuerpo que viaje a velocidad v describa una curva de radio R , esta componente tiene que ser igual a:

$$F_c = m \frac{v^2}{R}$$

- **Naturaleza:** Para un coche es el rozamiento con el suelo, para un tren la fuerza que ejerce el raíl, para un avión la fuerza del aire sobre los alerones, para un satélite la gravedad. `gravitacion04.py`

Relevancia en Videojuegos

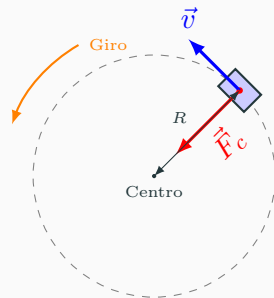
Sin esta fuerza, los objetos seguirían trayectorias rectas por inercia. Es vital para simular el agarre en curvas o las órbitas en simuladores espaciales.

Aplicación en Videojuegos: Steering y Curvas

Para lograr una simulación realista, el motor gestiona el equilibrio entre la inercia lineal del cuerpo y la fuerza lateral que permite realizar el giro.

- **Derrape:** Si la F_c necesaria es mayor que la fricción disponible, el coche pierde el agarre lateral.
- **Cámaras:** Se usa para calcular trayectorias orbitales suaves y fluidas alrededor del jugador.
- **Estabilidad:** Una F_c aplicada muy arriba o muy abajo puede provocar el vuelco del vehículo.

$$\text{Condición de agarre: } m \frac{v^2}{R} \leq F_r$$

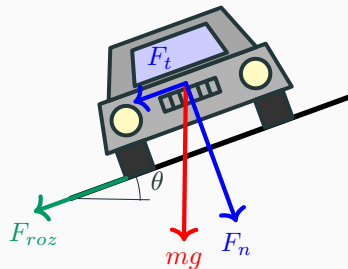


`fcentripeta01.py`
`parabolica_monza.py`

Peralte

- Es la **inclinación transversal** de la calzada en las curvas de una carretera o vía férrea.
- Se define por el ángulo θ respecto a la horizontal.
- Añade una componente del peso a la fuerza centrípeta

$$F_c = F_{roz} + F_t = F_{roz} + mg \sin \theta$$



- **Aumento de velocidad crítica:** Permite que los vehículos tomen curvas a mayores velocidades sin depender exclusivamente del rozamiento neumático-asfalto.
- **Seguridad:** Evita el **deslizamiento** hacia el exterior de la curva en condiciones de baja adherencia (lluvia o hielo), ya que la propia geometría de la vía ayuda a mantener el coche en trayectoria.
- **Confort:** Reduce la sensación de empuje lateral para los pasajeros.

¿Fuerza Centrífuga? El Observador Interno

Desde dentro de un vehículo en una curva, percibimos una fuerza que nos empuja hacia el exterior. ¿Es una fuerza real?

- **La Realidad (Sistema Inercial):** Un observador externo ve que tu cuerpo intenta seguir en línea recta debido a la **Inercia**. Es el coche el que, al girar, “choca” contra ti para obligarte a curvar.
- **La Percepción (Sistema No Inercial):** Si situamos el sistema de referencia dentro del coche (que está acelerando), para que las leyes de Newton sigan funcionando debemos añadir una **Fuerza Ficticia** o de inercia: la **Fuerza Centrífuga**.

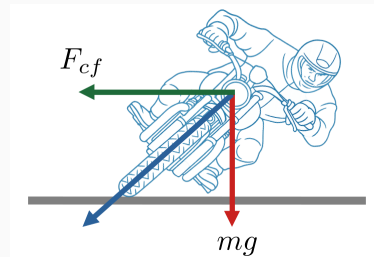
Fuerza Centrífuga ($\vec{F}_{cf}$)

Es una fuerza aparente, de igual magnitud y sentido opuesto a la centrípeta, que solo existe para el observador que gira:

$$F_{cf} = m \frac{v^2}{R}$$

Las curvas en Moto GP: Inclinación y Equilibrio

- Para que un motorista no vuelque en una curva, debe inclinar la moto un ángulo ϕ . Desde su sistema de referencia, busca el **equilibrio de fuerzas**:
- **Condición de Estabilidad** La fuerza **Resultante** ($\vec{R} = \vec{P} + \vec{F}_{cf}$) debe pasar exactamente por la línea que une el CM y el punto de contacto de los neumáticos.
- **Ángulo óptimo:** $\tan(\phi) = \frac{F_{cf}}{mg} = \frac{v^2}{gR}$.
- Si se inclina menos de lo debido para su velocidad, la resultante cae "fuera" del apoyo y la moto tiende a levantarse (subviraje); si inclina demasiado se cae.
- Explicar esto sin fuerzas ficticias es complicado (se puede, por supuesto)



Resultante alineada con el punto de apoyo.

3 Centro de masas

Rotación y centro de masas

La dinámica de rotación para una **masa puntual** no presenta demasiada complicación.

- La componente tangencial de la fuerza puede modificar la velocidad angular.
- La componente normal puede aumentar o disminuir el radio de giro.

Sin embargo, los cuerpos que simulamos no son puntuales, poseen **volumen y distribución de masa**.

- Los hemos tratado como partículas porque hemos aplicado siempre las fuerzas en el **centro de masas**: Punto donde puede considerarse concentrada toda la masa del cuerpo para describir su movimiento traslacional.
- **Centro de gravedad** Punto donde actúa la resultante de las fuerzas gravitatorias sobre el cuerpo.
 - En un campo gravitatorio **uniforme**, ambos puntos coinciden.
 - En general pueden diferir si el campo gravitatorio no es uniforme.

Cálculo del centro de masas

- A partir de ahora supondremos que el centro de masas y el centro de gravedad coinciden.
- **Sistema discreto de masas.** Si el cuerpo está formado por partículas puntuales m_i situadas en $\vec{r}_i$,

$$\vec{r}_{CM} = \frac{1}{M} \sum_i m_i \vec{r}_i, \quad M = \sum_i m_i$$

- **Distribución continua de masa.** Para un cuerpo con densidad $\rho(\vec{r})$,

$$\vec{r}_{CM} = \frac{1}{M} \int \vec{r} \rho(\vec{r}) dV, \quad M = \int \rho(\vec{r}) dV$$

- Es una **posición media ponderada por la masa**.
- Describe el movimiento traslacional del cuerpo como si toda su masa estuviera concentrada en ese punto.

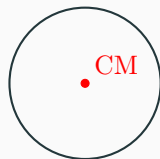
Centro de masas en pymunk

En `pymunk` los cuerpos rígidos se modelan mediante **figuras planas** (2D). Si asumimos **densidad uniforme**, el centro de masas coincide con el **centro geométrico** de la figura cuando este está claramente definido.

- Círculo, cuadrado o rectángulo, elipse, ...
- En estos casos el centro de masas se sitúa en ese punto:

$$\vec{r}_{CM} = \vec{r}_{\text{geom}}$$

- Esto simplifica los cálculos de simulación: el movimiento traslacional se puede tratar como si toda la masa estuviera concentrada en $\vec{r}_{CM}$, y la rotación se describe respecto a este punto.



Círculo



Rectángulo

En `pymunk` el origen de coordenadas dentro del cuerpo siempre se toma en el CM

Centro de masas de un triángulo

Si la **densidad es uniforme** coincide con el baricentro del triángulo

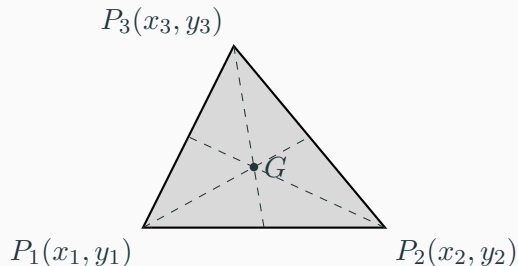
Triángulo de vértices

$P_1(x_1, y_1), P_2(x_2, y_2), P_3(x_3, y_3)$

- El **baricentro** G es el *punto de intersección de las medianas*.
- Se calcula promediando las coordenadas de los vértices:

$$x_{CM} = \frac{x_1 + x_2 + x_3}{3}, \quad y_{CM} = \frac{y_1 + y_2 + y_3}{3}$$

- El baricentro divide cada mediana en proporción 2:1 desde el vértice hasta el punto medio del lado opuesto.



Centro de masas de un polígono convexo

Polígono convexo: Vértices $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ enumerados en orden.

Área del polígono:

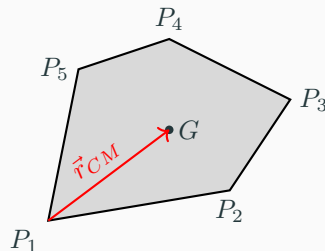
$$A = \frac{1}{2} \sum_{i=1}^n (x_i y_{i+1} - x_{i+1} y_i), \quad P_{n+1} = P_1$$

Centro de masas (centroide):

$$x_{CM} = \frac{1}{6A} \sum_{i=1}^n (x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i)$$

$$y_{CM} = \frac{1}{6A} \sum_{i=1}^n (y_i + y_{i+1})(x_i y_{i+1} - x_{i+1} y_i)$$

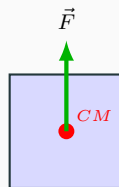
- Fórmulas válidas para cualquier polígono convexo o simple.
- El centroide es el centro de masas si **si el polígono tuviera densidad uniforme**.



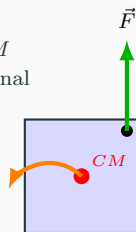
Efecto de la fuerza sobre el centro de masas

La **inercia del cuerpo** actúa sobre el centro de masas:

- Si aplicamos una fuerza sobre el $\vec{r}_{CM}$, el cuerpo se desplaza como una **masa puntual**.
- Si la fuerza se aplica sobre otro punto el cuerpo gira.
- En Pymunk, para aplicar una fuerza en el CM :
`body.apply_force_at_local_point(force, point=(0,0))`
- Para $(x, y) \neq (0,0)$ la fuerza se aplica en el punto de coordenadas (x, y) **con respecto al CM**



Fuerza sobre el CM
Movimiento traslacional

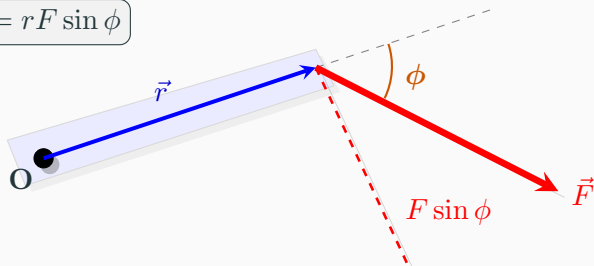


Fuerza fuera del CM
Traslación + Rotación

Momento de una Fuerza (Torque)

El **torque** ($\vec{\tau}$) representa la capacidad de una fuerza para producir una rotación. Depende de la intensidad de la fuerza, como de su punto de aplicación y del ángulo que forma la fuerza con la línea que une el punto de aplicación y el centro de giro.

$$\tau = rF \sin \phi$$



Interpretación Física

Solo la componente de $\vec{F}$ perpendicular a $\vec{r}$ ($F \sin \phi$) genera giro. Si la fuerza apunta directamente al origen ($\phi = 180^\circ$), el torque es nulo.

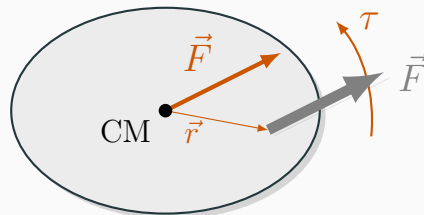
Descomposición: Fuerza neta vs. Torque

Al aplicar una fuerza $\vec{F}$ en un punto $\vec{r}$ alejado del CM , el efecto se descompone en:

- **Traslación:** La fuerza actúa sobre el CM ($\vec{a} = \vec{F}/m$).
- **Rotación:** Se genera un torque neto que cambia la velocidad angular.

- **Principio de Equivalencia**

- $\vec{F}$ excéntrica $\equiv \vec{F}$ en el CM + Torque
- Mayor $|\vec{r}|$, mayor efecto de giro (mayor “brazo de palanca”).



Implementación en Pymunk

`fcentripeta02.py`

Cuando usamos `body.apply_force_at_local_point(force, point)`, Pymunk calcula internamente:

$$\tau = \vec{r} \times \vec{F}$$

4 El Sólido Rígido. Momento de inercia

El Sólido Rígido

Sólido Rígido. Es un sistema de partículas donde las distancias entre ellas permanecen constantes:

$$|\vec{r}_i - \vec{r}_j| = \text{constante}, \quad \forall i, j$$

- **Consecuencia:** No hay deformaciones. El objeto mantiene su forma intacta tras una colisión.
- **Estado:** El movimiento total se descompone en la **traslación del CM** y la **rotación respecto al CM**.
- **Simplicidad:** En Pymunk, esto nos permite definir una única velocidad angular **body.angular_velocity** para todo el cuerpo.

La Resistencia al Giro: Momento de Inercia (I)

Si la masa es la oposición al cambio de movimiento lineal ($F = ma$), el **momento de inercia** es la oposición al cambio de rotación.

Definición para un sólido rígido Depende de cómo se distribuye la masa respecto al eje de giro:

$$I = \sum m_i r_i^2 \simeq \int r^2 dm$$

- **Distribución:** A igual masa, un objeto con la masa alejada del centro (ej. un aro) cuesta más de hacer girar que uno concentrado (ej. un disco).
- **Unidades:** $[I] = \text{kg} \cdot \text{m}^2$ (En **pymunk** $\text{kg} \cdot \text{px}^2$)
- **Ejemplo:** Un una rueda tarda más en caer por un plano inclinado que una esfera **aro_esfera.py**. Sin embargo, si suben una pendiente con la misma velocidad inicial, el aro tarda más en pararse **aro_esfera_subida.py**

Segunda Ley de Newton para la Rotación

Relacionamos la causa del giro (**Torque**) con el efecto cinemático (**Aceleración angular**) a través del **Momento de Inercia** (I).

Ecuación Fundamental

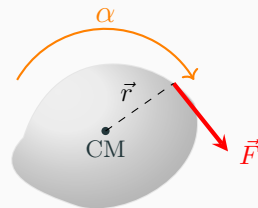
Análoga a $F = ma$ para la traslación:

$$\sum \tau = I\alpha$$

- **Causa** (τ): Torque neto sobre el sólido.
- **Oposición** (I): Resistencia al cambio de rotación.
- **Efecto** (α): Cambio en la velocidad angular ω .

En el motor de física

El motor resuelve en cada *step*: $\alpha = \frac{\sum \tau}{I}$. Integrando para actualizar ω y el ángulo θ .



El torque $\vec{\tau} = \vec{r} \times \vec{F}$ genera la aceleración α .

Analogía: Traslación vs. Rotación

Para facilitar la implementación, podemos ver la rotación como el “reflejo” angular de la traslación:

Magnitud	Lineal	Angular
Causa	Fuerza (F)	Torque (τ)
Oposición	Masa (m)	Inercia (I)
Estado	Velocidad (v)	Vel. angular (ω)
Cambio	Aceleración (a)	Acel. angular (α)
Ley de Newton	$F = ma$	$\tau = I\alpha$

Nota: En Pymunk, `body.torque` acumula todas las fuerzas excéntricas aplicadas antes de cada paso de simulación.

Momentos de Inercia de Figuras Comunes

Figura		Fórmula
Anillo delgado (aro, cilindro hueco)		mR^2
Arandela, cilindro con espesor		$\frac{1}{2}m(R_1^2 + R_2^2)$
Cilindro macizo (disco)		$\frac{1}{2}mR^2$
Esfera sólida		$\frac{2}{5}mR^2$
Corteza esférica		$\frac{2}{5}m \frac{R_2^5 - R_1^5}{R_2^3 - R_1^3}$
Esfera hueca		$\frac{2}{3}mR^2$

Figura		Fórmula
Anillo (Eje diámetro)		$\frac{1}{2}mR^2$
Disco (Eje diámetro)		$\frac{1}{4}mR^2$
Cilindro (Eje central $\perp$)		$\frac{1}{4}mR^2 + \frac{1}{12}mL^2$
Varilla (Eje central $\perp$)		$\frac{1}{12}mL^2$
Placa Rectangular (Eje central $\perp$)		$\frac{1}{12}m(a^2 + b^2)$

Teorema de Steiner (Ejes Paralelos)

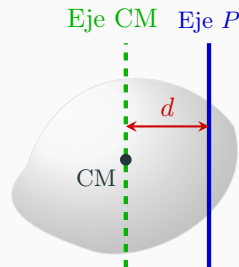
Este teorema permite calcular el momento de inercia I de un sólido rígido respecto a un eje cualquiera, siempre que sea paralelo a un eje que pase por su centro de masas (I_{cm}).

$$I = I_{cm} + md^2$$

- I_{cm} : Momento de inercia respecto al eje del centro de masas.
- m : Masa total del cuerpo.
- d : Distancia perpendicular entre ejes.

Utilidad

Fundamental para sistemas donde el eje de giro está desplazado, como en péndulos físicos o manivelas. Siempre se cumple $I > I_{cm}$.



El momento de inercia aumenta con el cuadrado de la distancia al CM.

Cálculo de Inercia en Pymunk (2D)

Pymunk utiliza fórmulas analíticas para calcular el momento de inercia escalar. Se basa en funciones auxiliares que integran sobre la masa del objeto.

```
1 mass = 1.0
2 # Inercia para un circulo solido
3 inertia_circ = pymunk.moment_for_circle(
4     mass,0,40)
5 # Inercia para un chasis rectangular
6 inertia_box = pymunk.moment_for_box(
7     mass, (160, 40))
8 # Inercia para un poligono convexo (ej.
9     triangulo)
10 # Los vertices se definen relativos al
11     centro de masas
12 vertices = [(-20, -20), (20, -20), (0, 30)]
inertia_poly = pymunk.moment_for_poly(mass,
    vertices,offset=(0,0),radius=0)
```

- La inercia es un único valor float (I_z).
- Los vértices deben definirse **secuencialmente** en sentido horario o antihorario.
- **Convexidad:** Solo es válido para polígonos convexos.
- **offset** sirve para calcular I respecto de un punto desplazado (Utiliza T. de Steiner).
- **radius** para tener en cuenta esquinas redondeadas.

Relación entre Body, Shape e Inercia

En Pymunk, el **body** no tiene dimensiones: es un punto con masa y momento de inercia. La geometría la define el **shape**.

- **Círculos y Cajas:** El **body** se sitúa por defecto en el centro de la figura. El momento de inercia se calcula automáticamente respecto a ese centro.
- **Polígonos:** El **body** se ubica en el punto (0,0) desde el que se han dado las coordenadas.

Ejemplo: Si definimos un triángulo como [(0,0), (10,0), (5,10)], el eje de giro será (0,0) y no su centro de masas (CM). **¿Cómo se corrige?**

```

1  vertices = [(0,0), (10,0), (5,10)]
2  # Forma compacta de corregir el posicionamiento:
3  centroide = pymunk.Poly(pymunk.Body(), vertices).center_of_gravity
4  vertices = [p - centroide for p in vertices]
5  # Ahora la inercia se calcula respecto al CM real
6  inercia = pymunk.moment_for_poly(masa, vertices)
7  cuerpo = pymunk.Body(masa, inercia)
8  forma = pymunk.Poly(cuerpo, vertices)

```

`inercia_rectangulo.py`

Definición de Inercia en PyBullet (URDF)

En PyBullet, el tensor de inercia se define explícitamente en el archivo URDF. Si no se especifica, el motor intenta calcularlo a partir de la geometría de colisión.

```
1  <link name="chasis">
2  <inertial>
3  <origin xyz="0 0 0" rpy="0 0 0"/>
4  <mass value="5.0"/>
5  <inertia ixx="0.1" ixy="0" ixz="0"
6  iyy="0.2" iyz="0"
7  izz="0.2"/>
8  </inertial>
9  </link>
10
```

- Los valores **ixx**, **iyy**, **izz** son los momentos principales.
- Los valores **ixy**, **ixz**, **iyz** son los productos de inercia.

Representación del Tensor de Inercia

PyBullet utiliza una matriz simétrica 3×3 para representar cómo se distribuye la masa en el espacio tridimensional:

$$\mathbf{I} = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix}$$

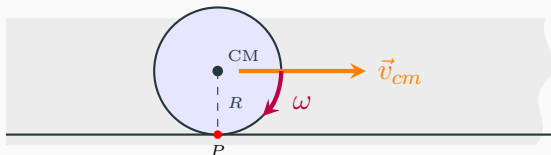
- **Diagonalización:** PyBullet diagonaliza esta matriz para encontrar los ejes principales.
- **Estabilidad:** Valores de inercia incorrectos o nulos pueden causar que el integrador numérico explote (simulación inestable).

5 Rodadura y deslizamiento

Cinemática en plano horizontal: Rodadura Pura

Se dice que un cilindro de radio R realiza una **rodadura pura** cuando el punto de contacto con el suelo tiene velocidad instantánea nula ($\vec{v}_p = 0$).

Condición de rodadura pura: $v_{cm} = \omega R$



- El centro de masas avanza a una velocidad v_{cm} .
- El cuerpo rota con velocidad angular ω .
- En rodadura pura, la distancia recorrida por el CM coincide con la longitud de arco desenrollada: $s = \theta R$.

Estados de deslizamiento y rotación

Cuando la relación $v_{cm} = \omega R$ no se cumple, el cuerpo experimenta deslizamiento. Podemos distinguir tres casos críticos:

Estado	Relación	Efecto visual
Deslizamiento puro	$\omega = 0, v > 0$	El cuerpo “bloqueado” desliza.
Rotación pura (Patina)	$v = 0, \omega > 0$	Rueda girando en el sitio.
Rodadura con deslizamiento	$v \neq \omega R$	El punto P tiene velocidad relativa.

La velocidad del punto de contacto es $\vec{v}_p = \vec{v}_{cm} + \vec{\omega} \times \vec{r}_{p/cm}$. En magnitud, para el punto más bajo:

$$v_p = v_{cm} - \omega R$$

Importante: cuando hay rodadura pura la fuerza de rozamiento solo produce la rotación, **no disipa energía** puesto que no hay movimiento.

¿Cuándo “patina” la rueda?

El término “patinar” o “derrapar” ocurre cuando hay una discrepancia entre la tracción y el avance:

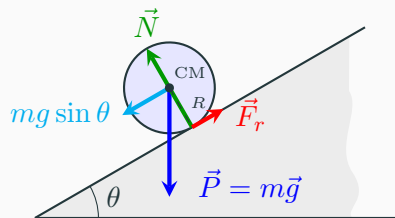
- **Deslizamiento por exceso de velocidad** ($v_{cm} > \omega R$): Ocurre en una frenada brusca con las ruedas bloqueadas. El punto de contacto se arrastra hacia adelante respecto al suelo. `bola_bolos.py`
- **Patinaje por exceso de torque** ($v_{cm} < \omega R$): Ocurre al acelerar violentamente sobre una superficie de baja fricción (hielo o barro). La rueda gira más rápido de lo que el vehículo avanza; el punto de contacto se desplaza hacia atrás respecto al suelo. `rueda02.py`, `coche01.py`

La Fuerza de Rozamiento F_r

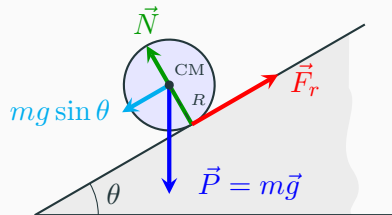
En rodadura pura, la F_r es estática. En el momento en que patina, la F_r pasa a ser cinética ($\mu_k N$), lo que reduce drásticamente el control sobre el movimiento.

Dinámica de un cilindro en plano inclinado

Para un cilindro de masa m y radio R que rueda sin deslizar en un plano de inclinación θ :



Deslizamiento



Rodadura pura

- **Componente tangencial** ($mg \sin \theta$): impulsa el movimiento.
- **Normal** ($\vec{N}$): Reacción del plano ante $mg \cos \theta$.
- **Fuerza de rozamiento** ($\vec{F}_r$): Estática, genera el torque $\tau = F_r R$.

Ecuaciones de Newton-Euler con F_r

El sistema se resuelve vinculando el movimiento lineal y el angular:

$$Mg \sin \theta - F_r = Ma_{cm}$$

$$\tau = F_r R = I\alpha$$

Bajo **rodadura pura** ($a_{cm} = \alpha R$), la aceleración general es:

$$a_{cm} = \frac{g \sin \theta}{1 + \frac{I}{MR^2}}$$

La fricción estática no realiza trabajo ($v_P = 0$). **La energía mecánica se conserva**, transformándose de potencial a cinética (traslación + rotación).

Conclusión: A menor momento de inercia tenemos la masa más concentrada en el centro, cuesta menos hacerla girar por lo que tenemos **mayor aceleración lineal** (que está ligada a la angular). [aro_esfera.py](#), [aro_esfera_subida.py](#).

Aceleración según el cuerpo:

- **Esfera Sólida** ($I = \frac{2}{5}MR^2$):

$$a_{cm} = \frac{5}{7}g \sin \theta \simeq 0,71g \sin \theta$$

- **Cilindro Sólido** ($I = \frac{1}{2}MR^2$):

$$a_{cm} = \frac{2}{3}g \sin \theta \simeq 0,67g \sin \theta$$

- **Cilindro Hueco** ($I = MR^2$):

$$a_{cm} = \frac{1}{2}g \sin \theta = 0,50g \sin \theta$$

Snippet: Aplicando Torque en Pymunk

Para hacer que un objeto rote en un simulador 2D:

```
1  import pymunk
2  # Crear cuerpo y forma
3  # El segundo parametro es el momento de inercia
4  body = pymunk.Body(mass=1, moment=10)
5  # Aplicar torque directamente (en Newton-unidades)
6  body.torque = 500
7  # O aplicar una fuerza excentrica
8  # Fuerza de 100 en el eje Y, aplicada a 20 unidades del centro
9  # Esto genera un torque automatico: tau = r * F
10 body.apply_force_at_local_point((0, 100), (20, 0))
11
```

Recuerda: El momento de inercia es la “resistencia” a este torque. En Pymunk, si el cuerpo es estático, el momento de inercia debe ser infinito (`float('inf')`).

6 Orientación en 3D

Ángulos de Euler (convenio de Tait-Bryan)

- Varias convenciones. En el tema 1 vimos la que se utiliza en Física Teórica ($Z-X-Z$).
- La más usada en Aviación, Robótica y PyBullet es el **convenio de Tait-Bryan**.

Convención Aeroespacial (XYZ)

- **Roll (Alabeo):** Eje X Longitudinal
- **Pitch (Cabeceo):** Eje Y Transv.
- **Yaw (Guiñada):** Eje Z Vertical

Limitaciones Críticas

- **Orden:** El resultado varía según la secuencia (XYZ , ZYX , etc.).
- **Gimbal Lock:** o Bloqueo de Cardán.
- **Importante:** Las rotaciones son **intrínsecas** (alrededor de los ejes móviles del objeto, no los ejes fijos del mundo).
- **La orientación total** es el producto de las matrices elementales (orden $X \rightarrow Y \rightarrow Z$):

$$R = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\phi)$$
- PyBullet no usa **Ángulos de Euler** internamente; son solo una interfaz humana. Utiliza **cuaterniones** ($[x, y, z, w]$) para evitar singularidades matemáticas.

¿Por qué evitar los Ángulos de Euler?

Aunque son intuitivos (Roll, Pitch y Yaw), presentan un fallo crítico en simulación física y animación: el **Gimbal Lock** (Bloqueo de cardán).

- **El Problema:** Al rotar un eje 90° , dos de los tres ejes de rotación pueden alinearse.
- **La Consecuencia:** Se pierde un grado de libertad. El motor de física no sabe cómo calcular la rotación intermedia y el objeto da “saltos” o gira erráticamente.
- **La Solución:** Usar una representación que no dependa de rotaciones sucesivas en ejes fijos.

El Bloqueo de Cardán (Singularidad Matemática)

La orientación total es el producto de las matrices elementales: $R = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\phi)$

1. Matriz Genérica

$$R = \begin{pmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{pmatrix}$$

2. Singularidad ($\theta = 90^\circ$)

$$R_{90^\circ} = \begin{pmatrix} 0 & \sin(\phi - \psi) & \cos(\phi - \psi) \\ 0 & \cos(\phi - \psi) & -\sin(\phi - \psi) \\ -1 & 0 & 0 \end{pmatrix}$$

- **Pérdida de un grado de libertad:** La matriz colapsada ahora solo depende de un parámetro: la diferencia $(\phi - \psi)$.
- **Ambigüedad:** Existen infinitas combinaciones de Roll y Yaw para la misma orientación física.
- **Consecuencia:** El software no puede interpolar trayectorias de forma única, provocando giros bruscos o bloqueos del objeto.

Más allá de Euler: Los Cuaterniones

- Aunque los ángulos de Euler son intuitivos, presentan el problema del “gimbal lock”. Para evitarlo, motores como PyBullet operan internamente con **cuaterniones**.
- Son vectores de 4D, $q = [x, y, z, w]$ cuyas componentes se obtienen a partir de:
 - Un vector unitario $\hat{u}$ que representa un eje de giro.
 - Un ángulo de giro θ

$$q = \left[u_x \sin \left(\frac{\theta}{2} \right), u_y \sin \left(\frac{\theta}{2} \right), u_z \sin \left(\frac{\theta}{2} \right), \cos \left(\frac{\theta}{2} \right) \right]$$

- **Sin singularidades:** Un único giro, no hay bloqueo de cardán.
- **Interpolación (SLERP):** Permiten transiciones suaves entre orientaciones, ideal para el movimiento del balón.
- **Normalización:** El cuaternión debe ser unitario: $\sqrt{x^2 + y^2 + z^2 + w^2} = 1$.

Uso de Cuaterniones en PyBullet

No es necesario calcular los 4 números a mano; el motor ofrece herramientas de traducción.

```
1  import pybullet as p
2  # 1. Definimos lo que entendemos (Euler en radianes)
3  euler = [0, 0, 3.14/4]
4  # 2. Traducimos al lenguaje del motor (Cuaternion)
5  # Retorna una lista [x, y, z, w]
6  quat = p.getQuaternionFromEuler(euler)
7  # 3. Aplicamos la orientacion estable
8  p.resetBasePositionAndOrientation(body_id, [0,0,0], quat)
9
```

Nota sobre el formato

PyBullet almacena el componente escalar w al final: $[x, y, z, w]$. Ten cuidado si importas datos de otros motores (como Unity) que pueden usar $[w, x, y, z]$.

Uso de cuaterniones en PyBullet

En PyBullet, casi todas las funciones de posición y orientación devuelven o aceptan cuaterniones en formato de lista de 4 elementos: `[x, y, z, w]`.

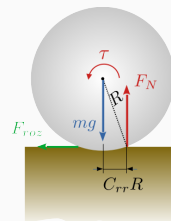
```
1  import pybullet as p
2  # Obtener posicion y orientacion actual
3  pos, quat = p.getBasePositionAndOrientation(body_id)
4  # Convertir de Cuaternion a Matriz de Rotacion (3x3)
5  matrix = p.getMatrixFromQuaternion(quat) # aqui no es necesario
6  # Rotacion relativa: Multiplicar cuaterniones
7  q_rot = p.getQuaternionFromAxisAngle([0, 0, 1], 3.14/4)
8  new_quat = p.multiplyTransforms([0,0,0], quat, [0,0,0], q_rot)[1]
9  # Aplicar la nueva orientacion
10 p.resetBasePositionAndOrientation(body_id, pos, new_quat)
11
```

7 Rozamiento por Rodadura

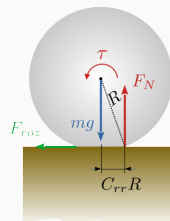
Deformación de los cuerpos que ruedan

A diferencia del rozamiento dinámico, el rozamiento por rodadura aparece debido a la deformación de los cuerpos en contacto.

- Se deforman el suelo, el cuerpo, o ambos.
- La normal ya no se aplica justo debajo del CM y aparece un torque que se opone a la rodadura.
- **Coefficiente de rozamiento por rodadura, C_{rr} :**
 - C_{rr} es adimensional.
 - Se interpreta como la **fracción del radio** que supone la distancia que se aparta de la vertical el punto de aplicación de $\vec{F}_N$.
 - Suele ser mucho menor que el rozamiento por fricción.



Se deforma el suelo



Se deforma el cuerpo

Se genera un **torque de frenado**:

$$\tau = C_{rr} \cdot R \cdot F_N$$

Implementación de Rodadura en Pymunk: Torque Manual

Aplicamos un torque que se opone al giro (ω) y cuya magnitud nunca supera el torque necesario para detener la rueda en un *step*.

```

1    tau_rod = 0.02 * R * (body.mass * 900)    #Crr=0.02 y  FN=mg
2    # 2. Torque para frenado total en un paso (I * w / dt)
3    tau_stop = (body.moment * abs(body.angular_velocity)) / (1/60.0)
4    # 3. Aplicar el minimo para evitar inversion del giro
5    if abs(body.angular_velocity) > 0.01:
6        freno = min(tau_rod, tau_stop)
7        body.torque -= (body.angular_velocity/abs(body.angular_velocity))*freno
8    else:
9        body.angular_velocity = 0 # Detencion limpia
10

```

- El torque es siempre opuesto a ω (siempre **disipativo**).
- $\min(t_{rod}, t_{stop}) \Rightarrow$ nunca puede empujar hacia atrás
- Podemos elegir C_{rr} según la superficie (arena, asfalto, etc.) o tipo de rueda.

Implementación de Rodadura en Pymunk: Torque Manual

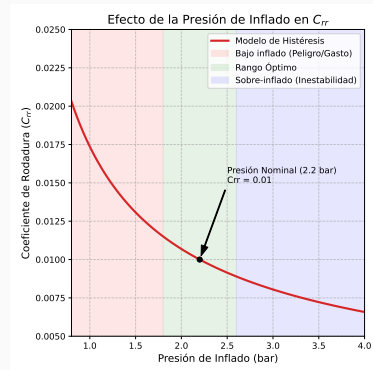
- Le damos al torque resistente el valor máximo $\tau = C_{rr} \cdot R \cdot F_N$
- Pero nunca permitimos que provoque giro hacia atrás.
- C_{rr} lo hemos expresado como una fracción del radio (e.g. 2 %)
- **IMPORTANTE:** Si F_N cambia (e.g. cambia la pendiente o aparece otra fuerza perpendicular a la superficie) hay que modificar a mano el valor del torque
- Sirve también para simular el rozamiento de un eje.

`rueda02.py`

`rueda_roz_rodadura.py`

Influencia de la Presión de Inflado en C_{rr}

- Una **baja presión** aumenta la deformación del neumático (histéresis).
- A mayor flexión de los flancos, mayor disipación de energía en forma de calor.
- $C_{rr} \propto \frac{1}{P^\alpha}$ con α entre 0.5 y 0.9.
- **Asfalto (Suelo duro):** El aumento de presión reduce el C_{rr} de forma asintótica, mejorando la eficiencia hasta alcanzar el límite de rigidez estructural.



- **Compromiso:** Un inflado excesivo reduce el rozamiento pero sacrifica la superficie de contacto (agarre) y la capacidad de absorción de impactos.
- **Impacto Energético:** Una caída del 20 % en la presión puede suponer un incremento del 10 % en la resistencia a la rodadura.

Influencia de la Presión de Inflado en C_{rr}

Rangos Típicos de α (Exponente de Sensibilidad)

- **0.7 – 0.8:** Estándar para neumáticos radiales modernos (turismos).
- **0.5 – 0.6:** Neumáticos de estructura diagonal o alta carga.
- **0.9:** Neumáticos de alta eficiencia o competición.

Interpretación del Valor

- α **Elevado:** Alta sensibilidad al inflado. Pequeñas pérdidas de presión penalizan drásticamente el consumo.
- α **Reducido:** Estructura rígida. El C_{rr} es más estable pero el neumático absorbe peor las irregularidades.
- **Nota sobre el Terreno:** Este modelo asume **suelo rígido**. En suelos deformables (arena/barro), α puede ser negativo (más presión $\rightarrow$ más hundimiento $\rightarrow$ más C_{rr}).

Implementacion en PyBullet (3D)

PyBullet permite definir la rodadura mediante la función `changeDynamics`. Actúa como un torque que modifica el cuaternión (rotación) en cada paso.

```
1  import pybullet as p
2  # Ejemplo con un balon
3  p.changeDynamics(
4      bodyUniqueId=ball_id,
5      linkIndex=-1,
6      lateralFriction=0.5,      # Friccion estandar (deslizamiento)
7      rollingFriction=0.01,    # Resistencia a rodar (e.g. cespced)
8      spinningFriction=0.001,  # Resistencia al giro (e.g. rozamiento eje)
9      restitution=0.8 )       # Rebote
10
```

- **rollingFriction**: Implementa el rozamiento por rodadura.
- Este valor se aplica solo durante el **contacto** con otra superficie.

Ejemplo Practico: Cálculo de parámetros

Si tenemos un balón de futbol ($R = 11$ cm) y estimamos una deformación del césped de $b = 1$ mm tendríamos un $C_{rr} = b/R = 0,01$.

- En **Pymunk** no hay que aplicar torque, basta con igualar el parámetro `rollingFriction` al valor de C_{rr} .
- **Torque Físico:** PyBullet aplicará un torque $\tau = C_{rr} \cdot R \cdot F_N$, recalculando F_N si es necesario.

```

1  # Calculo del coeficiente adimensional
2  b = 0.001  # 1 mm
3  R = 0.11   # 11 cm
4  Crr = b / R  # aprox 0.009.
5  p.changeDynamics(ball_id, -1, rollingFriction=Crr)
6

```

Conclusion: Un `rollingFriction` de 0,01 en PyBullet significa que la normal se desplaza un 1 % del radio del objeto hacia adelante.

Dinámica y movimiento para videojuegos y gamificación

Tema 4. Projectiles

Pedro Ángel Bernaola Galván

Febrero 2026

Contenido

- 1 Introducción
- 2 Modelo solo con gravedad
- 3 Resistencia del Aire
- 4 Variación con la altitud
- 5 Efecto del viento
- 6 Rotación. Efecto Magnus

1 Introducción

Introducción: ¿Qué es un Proyectoil?

En el desarrollo de videojuegos, un **proyectil** es cualquier objeto lanzado al espacio que solo está sujeto a fuerzas externas (gravedad, resistencia del aire, viento).

Ejemplos comunes en juegos:

- Balística clásica (balas, flechas, granadas, fase balística de misiles).
- Deportes (pelotas de golf, tenis o fútbol).
- Elementos del entorno (escombros tras una explosión).



El objetivo físico es predecir la **trayectoria** actualizando la posición y velocidad en cada frame.

Iremos añadiendo complejidad

Vamos a ir haciendo modelos cada vez más complejos añadiendo niveles de realismo creciente:

- **Modelo con solo gravedad:** Trayectoria parabólica ideal.
- **Modelo con Rozamiento (Drag):** Introducimos la resistencia del aire.
- **Modelo con Entorno:** Añadimos el efecto del viento.
- **Modelo con Rotación (Spin):** Efecto Magnus y sustentación.

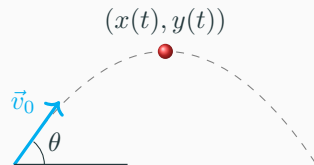
2 Modelo solo con gravedad

Tiene solución analítica

En el modelo ideal (sin rozamiento), no necesitamos simular paso a paso. Si conocemos las condiciones iniciales, la trayectoria está *perfectamente determinada* para cualquier instante de tiempo t .

Condiciones Iniciales ($t = 0$):

- Posición: (x_0, y_0)
- Velocidad: $v_{x0} = v_0 \cos \theta$, $v_{y0} = v_0 \sin \theta$



Ecuaciones del Movimiento:

Eje Horizontal (MRU)

$$v_x(t) = v_{x0}$$

$$x(t) = x_0 + v_{x0}t$$

Eje Vertical (MRUA)

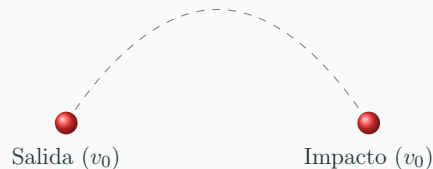
$$v_y(t) = v_{y0} - gt$$

$$y(t) = y_0 + v_{y0}t - \frac{1}{2}gt^2$$

Limitaciones del Modelo “Quick-and-Dirty”

Este sistema es el que usan la mayoría de juegos sencillos, pero carece de realismo por:

- **Simetría perfecta:** La velocidad de impacto es igual a la de salida.
- **Alcance exagerado:** En el vacío, una bala llegaría a distancias irreales.
- **Masa irrelevante:** Una pelota de ping-pong volaría igual que una bala de cañón.
- **Trayectoria en un plano:** No existen fuerzas laterales, sólo la gravedad hacia abajo.



3 Resistencia del Aire

Movimiento de un sólido dentro de un fluido

- Cuando un objeto se mueve dentro de un fluido aparece una fuerza que lo frena y que siempre es opuesta a la velocidad (**drag**).
- Hasta ahora, hemos visto el **rozamiento viscoso** (modelo de Stokes), donde la resistencia depende solo de la fricción interna del fluido.
- Sin embargo, en proyectiles reales aparece un **factor adicional** determinante:
 - **Resistencia por Viscosidad**: Domina en objetos microscópicos o fluidos muy densos (fricción superficial).
 - **Resistencia por Inercia**: El proyectil debe realizar un trabajo mecánico para “apartar” la masa del fluido y abrirse camino.



La fuerza total combina
fricción + colisiones

Diferencia clave: Mientras que el rozamiento viscoso es lineal con v , este nuevo factor inercial depende de v^2 .

Régimen Laminar y Turbulento

El **comportamiento del fluido alrededor del proyectil** se puede clasificar en dos categorías principales:

Regimen laminar

- Modelo de Stokes, $F_v \propto v$.
- Predominan las fuerzas viscosas.
- Velocidades bajas y fluidos muy viscosos (e.g. aceite).
- Objetos pequeños o microscópicos.

Regimen turbulento

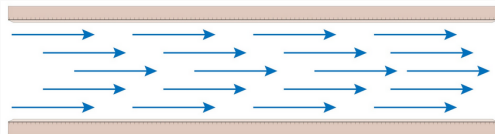
- Modelo de Newton, $F_v \propto v^2$
- Predominan las fuerzas de inercia, colisiones con las partículas. El objeto tiene que “apartarlas” para abrirse camino.
- Velocidades altas y fluidos poco viscosos (e.g. aire)

Régimen Laminar y Turbulento

En estos dos regímenes el movimiento de las partículas de fluido es muy diferente

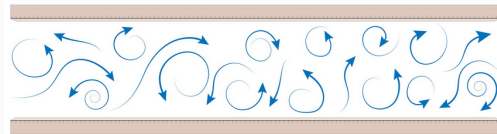
Regimen laminar

- Las partículas se mueven en “laminas” paralelas.
- El flujo es suave, predecible y las líneas de corriente no se cortan.



Regimen turbulento

- Movimientos caóticos y remolinos, presencia de vórtices.
- Fluctuaciones rápidas de presión y velocidad.



El Número de Reynolds en Sólidos

Para un sólido moviéndose en un fluido, Re mide la relación entre las fuerzas de inercia (el fluido se opone a que lo aparten) y las que se oponen al movimiento por rozamiento (viscosidad). Su valor **indica si estamos en régimen laminar o turbulento**

$$Re = \frac{\rho \cdot v \cdot L}{\mu}$$

ρ densidad del fluido, v velocidad

μ : **Viscosidad absoluta o dinámica**

$[\mu] = \text{Pa} \cdot \text{s} = \text{Poise}$

Longitud característica (L) depende de la geometría:

- **Esferas:** Diámetro de la esfera.
- **Perfiles alares:** Cuerda del ala.

El número de Reynolds refleja la competencia entre fenómenos de distinta dimensionalidad:

- **Fuerzas Viscosas** ($\propto L^2$): Fenómeno de superficie (fricción en la capa límite)
- **Fuerzas de Inercia** ($\propto L^3$): Asociadas a la masa y el volumen del fluido desplazado.
Dominan en grandes escalas.

Regimen Laminar

- **Modelo de Stokes** ($Re \ll 1$) : La viscosidad domina totalmente. No hay estela. Típico de microorganismos o gotas de niebla.
- **Ejemplo:** Una bola de 6 mm de diámetro cayendo por un recipiente lleno de aceite ($\mu \simeq 0.5 \text{ Pa}\cdot\text{s}$) a una velocidad típica de caída de 4 cm/s:

$$Re = \frac{\rho v L}{\mu} = \frac{900 \cdot 0,04 \cdot 0,006}{0,5} = 0,43 < 1$$

- La fuerza de rozamiento: $F_v = -6\pi\mu K r_e v$
 - μ es la viscosidad dinámica del fluido.
 - r_e es una dimensión característica (ej. radio de la esfera de igual volumen).
 - Es lo mismo que ya hemos visto con $b = 6\pi\mu K r_e$.
 - K es el factor de corrección de forma (adimensional).
 - Para la esfera $K = 1$

Factores de Forma y Viscosidad en Régimen Laminar

Geometría y Factor K

Cuerpo	Factor K
Esfera	1
Cubo	$\simeq 1,08$
Prisma Rect.*	$\simeq 1,14$
Cilindro $\parallel$	$\frac{2\pi L/d}{\ln(L/d) - 0,193}$
Cilindro $\perp$	$\frac{4\pi L/d}{\ln(L/d) + 0,5}$

*Para una relación de aspecto 1 : 1 : 2.

Viscosidad μ (Pa·s)

Fluido	μ
Aire	$1,8 \times 10^{-5}$
Agua	$1,0 \times 10^{-3}$
Aceite oliva	0,081
Glicerina	1,48
Miel	10,0

(Valores a $\simeq 20^\circ\text{C}$)

Régimen Intermedio

Régimen de Transición ($1 < Re < 10^3$) : Las viscosidad y la inercia tienen una importancia comparable. Comienza a aparecer una estela (**vórtices de Von Kármán**).

$$F_v = \frac{1}{2} C_d(Re) \rho A v^2$$

- **Ejemplo:** Gota de lluvia ($d \simeq 1,5$ mm) a una velocidad de 5 m/s: $Re \simeq 500$
- No existe una fórmula analítica sencilla como en Stokes.
- La fuerza depende del **cuadrado de la velocidad**.
- Aparece la densidad del fluido (ρ).
- A es el área de la sección transversal al flujo.
- C_d es el **coeficiente de arrastre** y es **función del número de Reynolds** (curvas empíricas casi siempre).
- Depende de la orientación del objeto.
- Una aproximación común para esferas es $C_d \simeq \frac{24}{Re} (1 + 0,15 Re^{0,687})$.

Régimen Turbulento

Modelo de Newton ($10^3 < Re < 2 \times 10^5$) : La inercia domina totalmente. Se forma una estela turbulenta tras el cuerpo. Projectiles, vehículos o deportistas.

$$F_v = -\frac{1}{2}C_d\rho Av^2$$

- **Ejemplo:** Pelota de tenis ($d \simeq 6,7$ cm), 180 km/h: $Re \simeq 2,2 \cdot 10^5$
- C_d depende de la forma pero es **constante** (no depende Re).
- Sigue dependiendo del cuadrado de la velocidad (v^2) y de la densidad (ρ).
- Para una esfera lisa, $C_d \simeq 0,44$ en este régimen.



Play video **1 2**

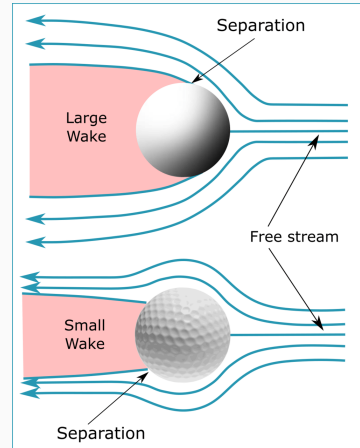
La Crisis de Arrastre ($2 \times 10^5 < Re < 3 \times 10^5$)

Existe un punto paradójico donde **aumentar la velocidad reduce drásticamente la resistencia.**

El fenómeno: A un Re crítico, la capa límite pasa de laminar a turbulenta. Aunque la turbulencia genera fricción, retrasa la **separación del flujo**, reduciendo el tamaño de la estela de succión.

El ejemplo de la Pelota de Golf:

- **Pelota lisa:** El flujo se separa pronto, creando una gran zona de baja presión detrás que la frena.
- **Pelota con hoyuelos (*dimples*):** Los hoyuelos inducen turbulencia prematura. Esta capa límite “excitada” tiene más energía cinética y logra rodear más superficie de la bola antes de separarse.

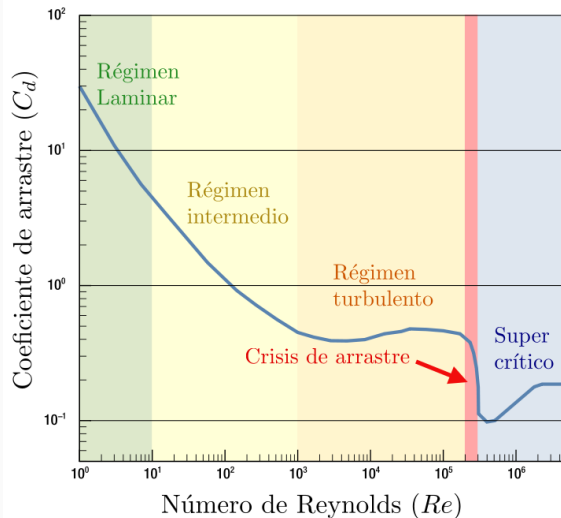


Régimen Supercrítico y Turbulento

Tras la crisis de arrastre ($Re \simeq 3 \times 10^5$), el coeficiente no sigue bajando indefinidamente. Entramos en el régimen **supercrítico**.

- **Recuperación del C_d :** El valor vuelve a subir un poco (e.g. para una esfera pasa de $\simeq 0,1$ hasta estabilizarse en torno a 0.2).
- **Independencia de Re :** A velocidades muy altas ($Re > 10^6$), el C_d se vuelve constante. La viscosidad ya no importa; solo manda la **geometría**.
- **Capa límite:** La turbulencia inyecta energía y retrasa la separación del flujo, manteniendo una estela estrecha.

Conclusión: Una vez que el flujo es totalmente turbulento, el coeficiente de arrastre se vuelve insensible a la velocidad.

C_d vs. Re para la esfera

- La **crisis de arrastre** y el **régimen supercrítico** son difíciles de conseguir: en el laboratorio o con diseños muy específicos (e.g. **bolas de golf**)
- Ante la duda lo mejor es aplicar el **régimen turbulento**, esto es $C_d \approx 0,44$.

Dependencia de la Forma Geométrica

Los valores de C_d entre 0.1 y 0.2 son específicos para la **esfera**. Para otros cuerpos, la geometría y la orientación dominan el arrastre.

- **Cuerpos Romos con Aristas** (Cubo, disco):
 - El flujo se separa **siempre** en las aristas vivas.
 - El C_d es casi constante ($\simeq 1,05 - 1,17$) y **no presenta crisis**.
- **Cuerpos Aerodinámicos** (Perfil alar):
 - Diseñados para retrasar la separación al máximo.
 - Pueden tener un C_d tan bajo como 0.04.

Nota: El arrastre depende de la **forma frontal** y de cómo esta obliga al aire a separarse de la superficie.

Coeficientes de arrastre de formas bidimensionales

- Los valores corresponden a un número de Reynolds $Re \simeq 1,0 \times 10^5$.
- Las imágenes indican la dirección del flujo de aire respecto a la geometría.
- La resistencia varía significativamente según la orientación de la forma cóncava/convexa.

Forma		C_D
Placa plana		2.0
Cilindro cuadrado		2.1
Cilindro cuadrado girado		1.6
Semicilindro (conv.)		1.7
Semicilindro (plano)		1.2
Cilindro circular		1.2
Cilindro elíptico 2:1		0.6
Semicilindro (cónc.)		2.3
Semicilindro (conv.)		1.2

Coeficientes de arrastre de formas tridimensionales

- Datos obtenidos para un número de Reynolds $Re \simeq 1,0 \times 10^5$.
- Se observa que las formas tridimensionales suelen presentar una **menor resistencia que sus versiones 2D equivalentes**.
- El flujo de aire puede rodear el objeto por más direcciones, reduciendo la zona de estela.

Forma		C_D
Placa cuadrada		1.17
Cubo (Cara plana)		1.05–1.07
Cubo girado		0.8–0.81
Hemisferio sólido		0.42
Cono 60°		0.5
Esfera		0.4–0.47
Elipsoide 2:1		0.27
Hemisferio (cónc.)		1.4
Hemisferio (conv.)		0.38–0.4

Regímenes Supersónico e Hipersónico

Cuando la velocidad del objeto (v) iguala o supera la velocidad del sonido (c), el aire deja de comportarse como un fluido incompresible.

La velocidad se mide con el **Número de Mach**: $M = \frac{v}{c}$

Régimen Supersónico ($M > 1$)

- Las ondas creadas por el avión se concentran delante (no pueden adelantarlos). Se produce una **onda de choque** (gradiente brusco de presión y temperatura).
- Aparece el **arrastre de onda** (*wave drag*), que consume mucha energía. Un nuevo factor (la compresión del fluido) que se suma a la viscosidad y la inercia.

Régimen Hipersónico ($M > 5$)

- El rozamiento con el aire lo calienta a miles de grados.
- **Efectos químicos**: El aire se disocia y se ioniza, convirtiéndose en **plasma**. La aerodinámica se mezcla con la termodinámica química.

Corrección por Compresibilidad: El Efecto Mach

A velocidades supersónicas ($v > 340$ m/s), el aire no se desplaza suavemente; se comprime formando una onda de choque que dispara el arrastre de forma no lineal.

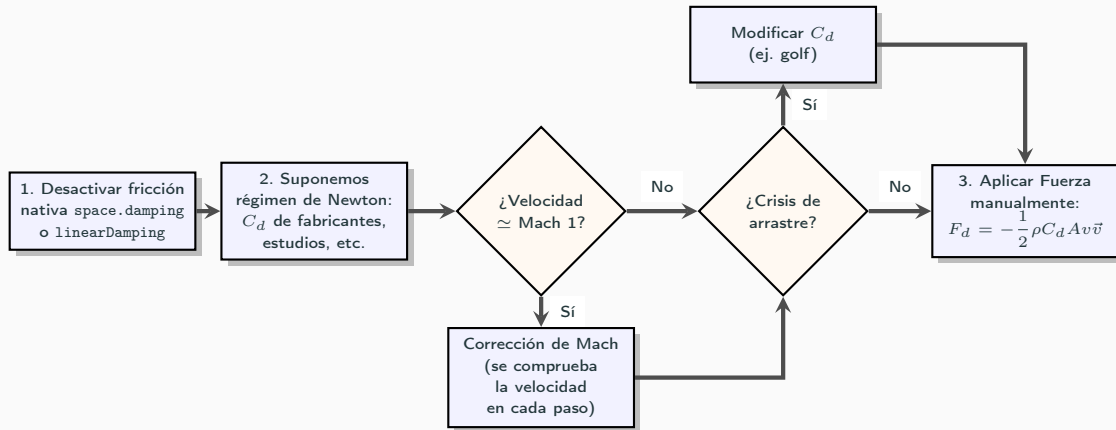
- **Arrastre de Onda:** Representa la energía disipada en la formación de la onda de choque frontal.
- **Factor de Corrección $\psi(M)$:** El C_d real se obtiene multiplicando el valor de Reynolds ($C_{d,Re}$) por un factor dependiente de Mach:

$$C_{d,total} = C_{d,Re} \cdot \psi(M)$$

Regímenes de Vuelo

- **Subsónico ($M < 0,8$):** $\psi(M) = 1,0$ (Sin efectos de onda).
- **Transónico ($0,8 \leq M < 1,2$):** $\psi(M) = 1,0 + 1,25(M - 0,8)$.
- **Supersónico ($M \geq 1,2$):** $\psi(M) = 1,5 + \frac{0,5}{M}$.

Flujo de Implementación: Fricción en Pymunk o PyBullet



* **Recordatorio:** La fuerza debe aplicarse manualmente sobre el cuerpo en cada paso de tiempo del simulador.

Comparativa de Implementación: Pymunk vs. PyBullet

Característica	Pymunk (2D)	PyBullet (3D)
Fricción Nativa	Ignorar <code>space.damping</code> (solo para contacto).	Poner <code>linearDamping</code> a 0 (evita modelo Stokes).
Modelo de Aire	Manual: Régimen de Newton ($\simeq v^2$).	Manual: Régimen de Newton ($\simeq v^2$).
Aplicación F	<code>apply_force_at_local_point</code> <code>apply_force_at_world_point</code>	<code>applyExternalForce</code>
Frecuencia	Flexible (basado en <code>step</code>).	Recomendado 240Hz (fijo).

Analogías

- La función nativa no sirve para balística real (solo rozamiento viscoso simple).
- La fuerza de arrastre F_d debe inyectarse en cada iteración del bucle. `enditemize`

Diferencias

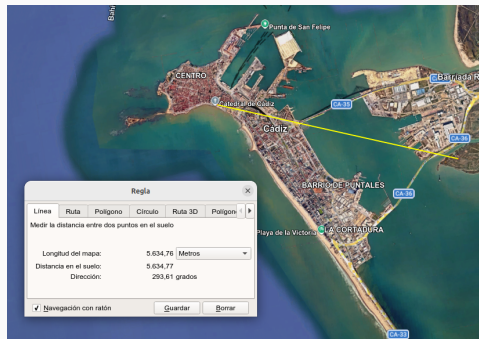
- **Bullet** es un motor basado en *impulsos*; es crítico recalcular la fuerza justo antes del paso físico.
- **Pymunk** es más permisivo con el sub-stepping manual.

* **Nota:** En ambos motores, si no se recalcula F_d en cada paso, la trayectoria perderá la curvatura real por la pérdida de energía.

El Desafío de Cádiz: Superando los Límites del Alcance

Para 1810, Cádiz era inexpugnable por tierra. El mariscal Soult ordenó alcanzar la ciudad desde el Trocadero, a una distancia de $\simeq 4000$ a 6000 metros, cuando los cañones estándar tenían un alcance eficaz de $\simeq 2000$ metros. [canion03.py](#)

- **La Solución:** Se fundieron en Sevilla los llamados “Granaderos de Villantroys”, morteros de diseño especial capaces de disparar proyectiles de gran calibre con ángulos de elevación extremos.
- **El Proyectil:** Obuses de 8 pulgadas rellenos de plomo para aumentar su densidad seccional y, por tanto, su coeficiente balístico (se pasa de una masa de 50 lb a 100 lb).
- **Carga de Proyección:** Se utilizaron cargas de pólvora masivas que ponían al límite la resistencia estructural del bronce.



Nota: A raíz del asedio de Cádiz se empezó a pensar en utilizar proyectiles ojivales.

4 Variación con la altitud

Dependencia de la densidad del aire con la altura

Para la troposfera ($h < 11000$ m), el gradiente térmico es $L \simeq 0,0065$ K/m (aprox. 2 grados por cada 300 m). La densidad se calcula según la ley de potencias:

$$\rho(h) \simeq \rho_0 \left(1 - \frac{Lh}{T_0}\right)^{\frac{g}{RL} - 1}$$

```

1  def get_rho(h):
2      rho0, T0 = 1.225, 288.15
3      L, R, g = 0.0065, 287.05, 9.80665
4      # Limitamos a la troposfera (11km)
5      h_corr = max(0.0, min(h, 11000.0))
6      temp_local = T0 - L * h_corr
7      exponente = (g / (R * L)) - 1
8      return rho0 * (temp_local / T0)**exponente
9

```

- h es la altitud
- ρ_0 y T_0 son la densidad y temperatura al nivel del suelo
- R es una constante que, para el aire seco, vale $R = 287,05$ J/(kg·K)

- **Precisión:** Captura el aumento de densidad por el enfriamiento del aire en altura.
- **Consistencia:** A 10000 m, la $T \simeq 223,15$ K (-50°C).

Dependencia de la densidad del aire con la altura

- La densidad ρ disminuye de forma no lineal con la altura.
- A 10,000 m, la densidad es $\simeq 35\%$ de la superficie.
- **Consecuencia:** Una bomba soltada a gran altura experimenta mucho **menos arrastre** inicial, ganando velocidad más rápido.
- El **alcance real** aumenta significativamente respecto al cálculo con densidad constante.

Ejemplo para 15°C en superficie

Altitud (m)	Temp. (°C)	Presión (kPa)	ρ (kg/m ³)
0	15.0	101.3	1.225
1,000	8.5	89.9	1.112
2,000	2.0	79.5	1.007
4,000	-11.0	61.7	0.819
6,000	-24.0	47.2	0.660
8,000	-37.0	35.7	0.526
10,000	-50.0	26.5	0.414
12,000	-56.5	19.4	0.312
15,000	-56.5	12.1	0.195

Dependencia de la velocidad del sonido con la temperatura

La velocidad del sonido en un gas ideal (como el aire) depende de su temperatura absoluta (T). A medida que el aire se enfría, las moléculas se mueven más lento, reduciendo la velocidad de transmisión de la onda.

$$c(T) \simeq \sqrt{\gamma \cdot R \cdot T}$$

```
1 def vel_sonido_temp(temp_k):  
2     # Constantes para el aire  
3     gamma = 1.4  
4     R = 287.05  
  
5  
6     # Velocidad en m/s  
7     c = math.sqrt(gamma * R * temp_k)  
8     return c  
9
```

- γ : Coeficiente de dilatación adiabática (1,4).
- R : Constante específica del aire seco (287,05 J/kgK).
- T : Temperatura absoluta en Kelvin (K).

Ejemplo: a 25°C, $c = 346,15$ m/s y a 0°C, $c = 331,32$ m/s

Dependencia de la velocidad del sonido con la altitud

Debido al gradiente térmico adiabático, la temperatura cae con la altura, lo que implica que el Mach 1 (velocidad del sonido local) disminuye a mayores altitudes.

$$c(h) \simeq \sqrt{\gamma \cdot R \cdot (T_0 - L \cdot h)}$$

```

1  def vel_sonido_altitud(alt_m,T0=288.15):
2      L = 0.0065
3      # Obtenemos T local (ISA)
4      y = max(0, min(alt_m, 11000))
5      t_local = T0 - L * y
6      # Usamos la funcion previa
7      return vel_sonido_temp(t_local)
8

```

Ejemplo: Si a nivel del mar $c = 346,15$ m/s entonces a 10000 m será $c = 306,08$ m/. Lo que al nivel del mar es 0.88 Mach, a 10000 es Mach 1.

Estas tres funciones se implementan en

`rozamiento_aire.py`

- T_0 : Temperatura a nivel del mar.
- L : Gradiente térmico (0,0065 K/m).
- h : Altitud sobre el nivel del suelo en metros.
- **Mach 1 no siempre es la misma velocidad**



Simulador Balístico B-52: Estructura del Código

Este código implementa una simulación en tiempo real que combina una interfaz de usuario en **Tkinter** con un motor de física 2D mediante **Pygame** y **Pymunk**.

- **Configuración:** Permite ajustar masa, diámetro y parámetros ambientales (C_d , densidad, Mach).
- **Arquitectura:** El bucle principal permite reiniciar la simulación pulsando ESC.
- **Cálculo de Fuerzas:** La función `aplicar_newton` es el núcleo físico. Calcula la fuerza de arrastre dinámica:

$$\vec{F}_{drag} = -\frac{1}{2}\rho(h) \cdot C_d(M) \cdot A \cdot |\vec{v}| \cdot \vec{v}$$

- Utiliza el `módulo rozamiento_aire` para obtener la densidad según la altitud y aplicar correcciones de compresibilidad cuando se activa la opción Mach.
- **Escalado:** Se utiliza una constante `M_PX` (18,05 m/px) para traducir las coordenadas de pantalla a magnitudes físicas reales.



`b52_02.py`

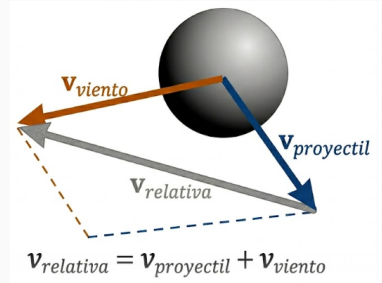
5 Efecto del viento

Efecto del viento. Velocidad Relativa

El viento **no añade una fuerza independiente**, sino que modifica la velocidad del proyectil respecto al aire ($\vec{v}_{rel}$), lo que a su vez redefine la fuerza de arrastre.

$$\vec{v}_{rel} = \vec{v}_{proyectil} - \vec{v}_{viento}$$

- Si el viento va en contra del movimiento la resistencia será mayor (mayor velocidad relativa)
- Si el viento va a favor se reduce el frenado y puede aparecer un **drag positivo**
- Por este motivo los aviones siempre despegan y aterrizan contra el viento.



Efecto del viento. Velocidad relativa

- Implementación en PyMunk

```
1 def calcular_v_relativa(v_p, v_w):
2     #v_p: [vx, vy] (proyectil)
3     # v_w: [vwx, vwy] (viento)
4     v_rel_x = v_p[0] - v_w[0]
5     v_rel_y = v_p[1] - v_w[1]
6     v_rel_mag = math.sqrt(v_rel_x**2 +
7                             v_rel_y**2)
8     return [v_rel_x, v_rel_y], v_rel_mag
9
```

- v_p : Velocidad del proyectil respecto a tierra.
- v_w : Velocidad del viento (horizontal).
- v_{rel} : Velocidad relativa, es la **velocidad efectiva que genera el rozamiento.**

Efecto del viento: Fuerza de Arrastre

La fuerza de arrastre (F_d) ahora se calcula usando el módulo de la **velocidad relativa**, y se aplica en **sentido opuesto a dicha velocidad relativa** para frenar o desviar el proyectil.

$$\vec{F}_d = -\frac{1}{2}\rho AC_d v_{rel} \vec{v}_{rel}$$

```

1 def fuerzas_viento(v_p, v_w, rho, A, Cd):
2     v_rel, v_rel_mag = calcular_v_relativa(v_p, v_w)
3     # Magnitud de la fuerza de arrastre
4     Fd = 0.5 * rho * A * Cd * v_rel_mag*v_rel
5     # Componentes de la fuerza (Newton)
6     fx = -Fd * (v_rel[0] / v_mag)
7     fy = -Fd * (v_rel[1] / v_mag)
8     return fx, fy
9

```

- ρ : Densidad del aire.
- A, C_d : Área frontal y coeficiente de arrastre.

Estabilidad Numérica: El límite de la Velocidad Relativa

Para evitar oscilaciones artificiales cuando el proyectil iguala la velocidad del viento, el simulador aplica un umbral de tolerancia:

- **Condición:** Si $|\vec{v}_p - \vec{v}_w| < 0,1 \text{ m/s} \rightarrow \vec{F}_d = 0$.
- **Efecto:** El proyectil queda “anclado” a la corriente de aire, moviéndose solidariamente con ella.
- **Realismo:** Este umbral previene el “ruido” numérico sin afectar a la precisión de la trayectoria macroscópica.

Nota: En este estado, el proyectil puede estarse moviendo muy rápido respecto al suelo, pero su aceleración aerodinámica es nula.

Implementación de F_d con aire en `rozamiento_aire.py`

Inestabilidad Aerodinámica: CG vs CP y Peso

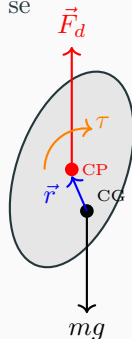
Cuando el Centro de Gravedad (CG) y el Centro de Presiones (CP) no coinciden, se genera un brazo de palanca y fuerzas contrapuestas.

Dinámica de Fuerzas y Torque:

- La fuerza de arrastre ($\vec{F}_d$) tira del CP , mientras que el peso (mg) actúa sobre el CG .
- Puede aparecer un **Torque Restaurador** (τ) alrededor del CG :

$$\tau = \vec{r}_{CP-CG} \times \vec{F}_d$$

- El peso (mg) actúa siempre verticalmente hacia abajo, siendo el CG el centro de giro del "péndulo".
 - El objeto oscila indefinidamente alrededor de la vertical, como un péndulo físico amortiguado por el aire.
 - Sin torque de amortiguamiento viscoso ($\tau \propto -\omega$), la oscilación puede amplificarse por sobrecorrección numérica.



6 Rotación. Efecto Magnus

Fundamento Físico: El Principio de Bernoulli

La Conservación de la Energía en Fluidos:

- En un fluido ideal, la suma de presión, energía cinética y energía potencial es constante.

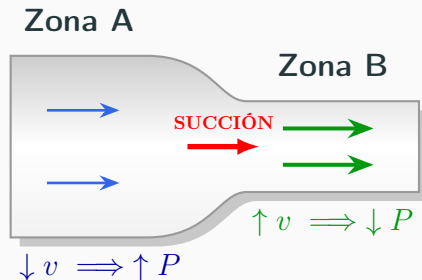
$$P + \rho gh + \frac{1}{2}\rho v^2 = \text{constante}$$

- Ecuación Simplificada** (sin cambio de altura):

$$P + \frac{1}{2}\rho v^2 = \text{constante}$$

- Si un término aumenta, el otro debe disminuir para mantener la igualdad

Conclusión: A mayor velocidad menor presión y viceversa



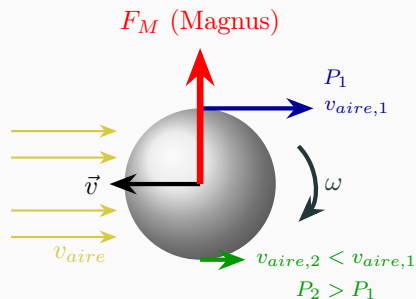
Ejemplo en una tubería: en el estrechamiento se aumenta la velocidad y se reduce la presión, produciendo succión hacia la derecha

Diferencia de presiones

Cuando el proyectil **gira sobre su propio eje (spin)**, el rozamiento con el aire crea una interacción curiosa:

- La fricción del proyectil arrastra el aire cercano creando una **capa límite** rotatoria.
- **Lado 1 (arriba):** La rotación se suma a la velocidad del aire. Mayor velocidad $\rightarrow$ Menor Presión.
- **Lado 2 (abajo):** La rotación se opone al flujo. Menor velocidad $\rightarrow$ Mayor Presión.

Conclusión: Se genera una fuerza neta hacia la zona de menor presión



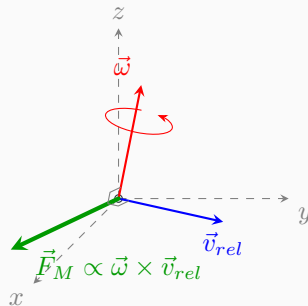
La fuerza Magnus

- La fuerza Magnus (F_M) es perpendicular a eje de rotación y perpendicular a la velocidad (**siempre la relativa con el aire**)

$$\vec{F}_M \propto \vec{\omega} \times \vec{v}_{rel}$$

- i.e. $\vec{F}_M$ lleva el sentido del vector unitario:

$$\hat{u}_m = \frac{\vec{\omega} \times \vec{v}_{rel}}{\omega v_{rel}}$$



En 2D (pymunk) es sencillo ya que $\vec{\omega}$ es siempre perpendicular a la pantalla:

- $\vec{F}_M$ a la **derecha** de la velocidad si $\omega > 0$, i.e. a favor de las agujas del reloj ⌚
- $\vec{F}_M$ a la **izquierda** de la velocidad si $\omega < 0$, i.e. en contra de las agujas del reloj ⌚

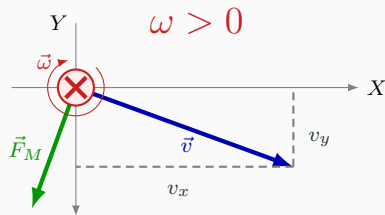
Criterio de signos en Pymunk/Pygame

Convención en Pymunk/Pygame:

- El eje Y crece hacia abajo.
- Para mantener un sistema ortogonal, **el eje Z positivo apunta hacia el interior de la pantalla.**

Interpretación de la velocidad angular (ω):

- Rotación *horaria* $\rightarrow$ Vector $\vec{\omega}$ en dirección $+Z$ (hacia adentro) $\rightarrow \omega > 0$.
- Rotación *antihoraria* $\rightarrow$ Vector $\vec{\omega}$ en dirección $-Z$ (hacia afuera) $\rightarrow \omega < 0$.



```

1  from pymunk import Vec2d
2  import numpy as np
3  if balon.velocity.length > 0:
4      u_m = Vec2d(-balon.velocity[1], balon.velocity[0])
5      u_m = np.sign(balon.angular_velocity) * u_m.normalized()
6  else:
7      u_m = Vec2d(0, 0)

```


Cálculo de la fuerza Magnus

La fuerza generada por el efecto Magnus se puede expresar como:

$$\vec{F}_M = \frac{1}{2} \rho v_{rel}^2 C_M A \hat{u}_m$$

El Coeficiente C_M :

- A diferencia del C_d , el C_M depende fuertemente de la **tasa de rotación** (S):

$$S = \frac{r \cdot \omega}{v}$$

Relación entre velocidad tangencial y lineal.

- Para proyectiles esféricos:

$$C_M \simeq \frac{kS}{2 + S}$$

Ejemplo de código: `magnus_futbol.py`

Importancia en Videojuegos

- Golf/Tenis:** El *backspin* alarga el vuelo; el *topspin* hace que caiga abruptamente.
- Volley:** El saque en salto con *topspin* hace que el balón vaya mucho más rápido hacia abajo



@sunray.beachvolley.

- Fútbol:** Tiros de falta con *rosca*.

Deporte	k
Golf	1.3
Tenis	1.1
Béisbol	0.9
Fútbol	0.7
Voleibol	0.6

Frenado Rotacional en Régimen de Newton

En fluidos de baja viscosidad y altas velocidades (aire), el frenado de un objeto giratorio no es lineal, sino que sigue una relación cuadrática con la velocidad angular ω .

- Aparece un **torque de resistencia** τ_{drag} que surge de la diferencia de presión y la fricción turbulenta en la superficie.

$$\tau_{drag} = -\frac{1}{2}\rho\omega^2 R^5 C_m$$

- ρ : Densidad del fluido (aire).
 - R : Radio del objeto (**muy importante**)
 - C_m : Coeficiente de momento rotacional (similar al C_d lineal).
- Este torque actúa **siempre en sentido opuesto al vector $\vec{\omega}$** , extrayendo energía cinética rotacional del sistema.
 - En **Pybullet** hay que implementarlo a mano. En **Pybullet** se puede modificar el atributo `angularDamping`: $0 \rightarrow$ sin freno, $1 \rightarrow$ frenado total

Valores de C_m según la Textura del Deporte

En el régimen de Newton ($Re > 10^4$), la rugosidad superficial es el factor que determina la pérdida de rotación.

Pelota	C_m (aprox.)	Naturaleza de la Superficie
Ping-pong	0,006	Plástico liso (mínima fricción).
Voleibol	0,015	Cuero sintético con juntas.
Fútbol	0,018	Paneles cosidos o termosellados.
Béisbol	0,035	Costuras de hilo prominentes.
Tenis / Pádel	0,085	Fieltro de lana (máximo agarre).

- El fieltro del **Tenis/Pádel** genera un torque de frenado casi 14 veces superior al del plástico del **Ping-pong**.
- En el **Voleibol**, el bajo C_m permite que los efectos laterales se mantengan estables durante toda la trayectoria de la pista.

Dinámica y movimiento para videojuegos y gamificación

Tema 5. Colisiones

Pedro Ángel Bernaola Galván

Febrero 2026

Contenido

- 1 Introducción
- 2 Conservación de la energía
- 3 Conservación del momento lineal y angular
- 4 Control de colisiones

1 Introducción

Las Colisiones

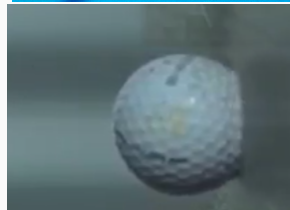
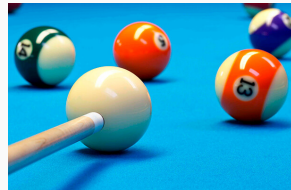
En el desarrollo de videojuegos, modelar el impacto entre objetos es fundamental para el realismo. Una colisión se define por:

- El intercambio de momento lineal y angular.
- La duración del impacto (habitualmente muy breve).
- La conservación o pérdida de energía cinética.
- El **Momento Lineal** ($\vec{p}$), y el **Momento Angular** ($\vec{L}$) se definen como:

$$\vec{p} = m \cdot \vec{v} \quad \vec{L} = I \cdot \vec{\omega}$$

donde m es la masa y $\vec{v}$ la velocidad del objeto e I y $\vec{\omega}$ el momento de inercia y la velocidad angular del objeto.

- $\vec{p}$ y $\vec{L}$ se conservan si no hay fuerzas exteriores
- **La energía no siempre se conserva**



`bloque_muelle.py`

La Complejidad de las Colisiones Reales

Implementar una colisión robusta desde cero es una tarea compleja que va más allá de aplicar leyes de conservación.

- **Detección de contacto:** Determinar el instante exacto y el punto de colisión (*collision detection*) requiere algoritmos geométricos costosos.
- **Gestión de la energía y el momento:** Además de conservar $\vec{p}$, hay que tratar la pérdida de energía (e) y la transferencia al momento angular.
- **Motores de física:** Afortunadamente, herramientas como Pymunk o PyBullet ofrecen una simulación de contacto muy superior a otros modelos (como el rozamiento del aire), evitando retoques manuales constantes.

2 Conservación de la energía

Coeficiente de Restitución (e)

Define cómo de **elástica** es una colisión. Se calcula mediante la relación de velocidades relativas antes y después del impacto:

$$e = -\frac{v_{2f} - v_{1f}}{v_{2i} - v_{1i}}$$

Valores típicos de e :

- $e = 1$: **Colisión perfectamente elástica** (se conserva la energía).
- $e = 0$: **Colisión perfectamente inelástica** (los objetos se pegan).
- $0 < e < 1$: **Colisiones reales** (fricción, deformación, calor).



Video. Cuna de Newton

`bolas01.py`

Elasticidad en Pymunk (Coeficiente de Restitución)

En Pymunk, la restitución se controla con la propiedad `elasticity` de los objetos `Shape`.

- Representa la capacidad de un objeto para conservar energía cinética tras un choque.
- Los valores se sitúan normalmente en el rango $[0, 1]$.
- **0.0 (Inelástico)**: El objeto no rebota (como plastilina).
- **1.0 (Perfectamente elástico)**: El objeto conserva toda su velocidad normal.
- El **coeficiente de restitución del choque** se define como el producto de las propiedades `elasticity` de los dos objetos.
- Se admite `elasticity > 1`: por ejemplo cuando se salta sobre una cama elástica
- Si la colisión es muy rápida, es conveniente hacer *substepping*

`bolas01.py`

`bloque_muelle.py`

Elasticidad en PyBullet

PyBullet gestiona la colisión elástica mediante el parámetro **restitution** en el método **changeDynamics**.

- **Asignación:** Se aplica a cada *link* o cuerpo de forma independiente.
- **Interacción:** A diferencia de otros motores, PyBullet requiere que **ambos** cuerpos definan una restitución > 0 para que el rebote sea visible.
- **Estabilidad:** Por defecto, el motor puede anular rebotes de baja energía para mantener la estabilidad del sistema (*contact damping*).
- **Paso de tiempo:** Para colisiones de alta velocidad y alta elasticidad, hay problemas igual que en Pymunk, se recomienda aumentar la frecuencia de muestreo: `p.setTimeStep(1/240.0)`

Problemas con el solapamiento de objetos

Debido a que el tiempo es discreto un cuerpo puede superponerse con otro antes de que se detecte la colisión. En el motor Pymunk, el mecanismo para resolver este problema se denomina **Bias**.

- **collision_bias**: Define qué porcentaje del error de solapamiento se corrige en cada paso. Si es muy alto, los objetos se separan de forma explosiva.
- **collision_slop**: Pymunk introduce además un “margen de tolerancia” (slop). Los objetos pueden penetrarse hasta esta distancia antes de que el *bias* empiece a actuar. Si es bajo, más precisión pero tiene mayor coste de cálculo y puede provocar vibraciones (correcciones continuas)

Implementación de `collision_slop` y `collision_bias`

Para gestionar la penetración, estos parámetros se ajustan en objeto `space`:

- `space.collision_slop` (El margen):
 - Si la penetración es menor al *slop*, el motor la ignora.
 - Ayuda a evitar que los objetos vibren (*jitter*) al estar en reposo.
- `space.collision_bias` (La corrección): Determina cómo de rápido se resuelve el solapamiento que supera al *slop*.
 - Es un porcentaje: 0,1 significa corregir el 10 % del exceso en cada paso.

Ejemplo:

```
1 space = pymunk.Space()
2 space.collision_slop = 0.1 # si pones 0.05, mas precision, menos margen
3 space.collision_bias = 0.8 # si pones 0.05, separacion mas rapida
4
```

ERP en PyBullet: El homólogo del *Bias*

PyBullet utiliza el **ERP** (*Error Reduction Parameter*), que conceptualmente cumple la misma función que el `collision_bias` de Pymunk.

- **Fracción de corrección:** Al igual que en Pymunk, el ERP determina qué parte de la penetración se resuelve por cada *step*.
- **Rango de valores:**
 - **0.0:** No se corrige la posición (los objetos se hunden).
 - **1.0:** Intento de corrección total en un solo *step* (causa inestabilidad y explosiones físicas).
 - **0.2 a 0.8:** Valores estándar para un equilibrio entre rigidez y estabilidad.
- **Estabilidad:** Se mantienen las mismas reglas: valores cercanos a 1.0 producen el mismo efecto de “explosión” o inestabilidad que vimos en Pymunk.

¿Dónde se cambia el ERP en PyBullet?

Diferencia de implementación: Mientras que en Pymunk suele ser un ajuste más global, en PyBullet el ERP es muy flexible y puede configurarse:

- A nivel global del motor físico para los contactos:
`p.setPhysicsEngineParameter(contactERP=0.8)`
- Individualmente para cada restricción (*cid*) o unión:
`p.changeConstraint(cid, erp=0.5)`
- **Contact ERP:** Un valor alto simula materiales muy rígidos (acero); un valor bajo simula contactos más “blandos” o permisivos.

3 Conservación del momento lineal y angular

Conservación del Momento Lineal

A partir de la Segunda Ley de Newton, $\vec{F} = \frac{d\vec{p}}{dt}$, si la fuerza externa neta sobre un sistema es nula ($\vec{F}_{ext} = 0$), entonces:

$$\frac{d\vec{p}_{tot}}{dt} = 0 \implies \vec{p}_{tot} = \text{constante}$$

Esta ley es universal: se cumple en cualquier colisión, independientemente de si es elástica o inelástica, ya que las fuerzas de interacción interna (Tercera Ley de Newton) siempre se cancelan por pares:

- “El momento lineal total de un **sistema aislado** permanece constante en el tiempo”.
- Esto implica que el centro de masas del sistema mantiene su estado de movimiento uniforme, ya que $\boxed{\vec{p}_{tot} = M\vec{v}_{cm}}$.

Ejemplo: Si me empujas, te empujo

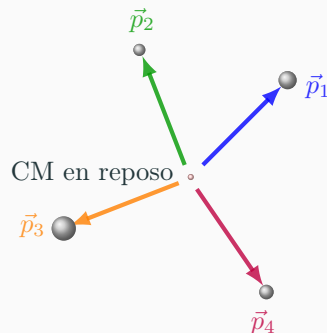
- Un coche acelerando *empuja* el suelo hacia atrás y el suelo lo empuja hacia delante (`coche_plataforma.py`)
- Cuando un objeto choca contra una pared ¿cuándo empuja más si el choque es elástico o inelástico? (`bola_pared.py`)
- ¿Qué pasa al chocar contra un objeto inamovible? (`bola_pared_infinita.py`)

¡¡CUIDADO!! Si es Pymunk el que gestiona la interacción, ya se encarga de que se cumpla la **Tercera Ley de Newton** y de que cada fuerza de *acción* tenga su correspondiente *reacción*. Si aplicamos fuerzas externas manualmente, tenemos que asegurarnos de que cumplimos esta condición.

Ejemplo: La Física de una Explosión

Consideremos una bomba inicialmente en reposo ($\vec{v}_{cm} = 0$).
Tras la explosión, las fuerzas internas proyectan fragmentos en todas direcciones:

- El momento total inicial es cero, por lo que la suma vectorial de los momentos de todos los fragmentos debe seguir siendo cero:
- $$\sum m_i \vec{v}_i = 0$$
- Los fragmentos se reparten de forma que el centro de masas del conjunto sigue en reposo en el mismo punto exacto de la explosión.
 - Si el conjunto se movía con velocidad rectilínea y constante, el CM también seguirá con velocidad rectilínea constante.



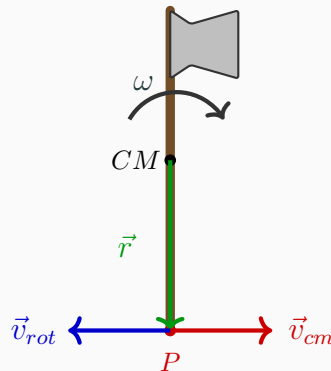
`cm.py`
`cm_parbola.py`
`baston.py`

Lanzamiento del hacha: El punto de reposo

Para que un punto P del hacha actúe como eje de giro (CIR), su velocidad en el sistema del laboratorio debe ser nula o igual a la velocidad de avance deseada.

$$\vec{v}_P = \vec{v}_{cm} + \vec{v}_{rot} \implies \vec{v}_{cm} = \vec{v}_P - (\vec{\omega} \times \vec{r}_{cm \rightarrow P})$$

- Si el mango está orientado hacia abajo ($r_y > 0$) y el giro es horario ($\omega > 0$).
- La rotación empuja el extremo hacia la **izquierda**.
- Por tanto, debemos **restar** ese vector (o sumar su opuesto) para que el CM absorba ese movimiento y el punto P parezca pivotar.

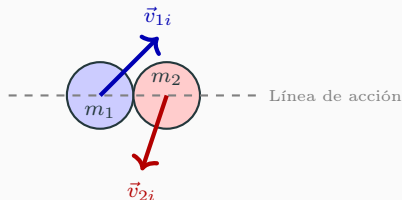


$$\text{Si } |\vec{v}_{cm}| = |\vec{v}_{rot}|, \vec{v}_P = 0$$

Colisiones Oblicuas (2D)

En la mayoría de los juegos, los objetos no chocan perfectamente centrados.

1. Identificar la **línea de acción** (el vector normal al contacto).
2. Descomponer las velocidades iniciales en componentes **normales** y **tangenciales**.
3. La colisión sólo afecta a las **componentes normales**.
4. Las componentes tangenciales permanecen constantes (si no hay fricción).



`colision_oblicua02.py`

Colisiones con Fricción y Rotación

Cuando un objeto choca con una superficie o con otro objeto, la fricción en el punto de contacto genera un **torque**. Este torque cambia el momento angular ($\vec{L}$) del objeto, haciendo que comience a girar o cambie su rotación previa.

- El cambio en el momento lineal tangencial depende del coeficiente de fricción estática y cinética.
- Si el impacto no es central, la fuerza genera una rotación instantánea.
- Además hay transferencia de momento lineal en una dirección distinta de la línea de acción

Ejemplo: si no hay fricción y una bola está inicialmente parada, después de la colisión se moverá en la dirección de la línea de acción. Si hay fricción esto ya no ocurre así.

`colision_oblicua02.py`

Momento Angular de un Sistema y su Conservación

Sistemas de Partículas Para un conjunto de masas, el momento angular total ($\vec{L}$) es la suma vectorial de los momentos individuales:

$$\vec{L} = \sum_i \vec{r}_i \times m_i \vec{v}_i$$

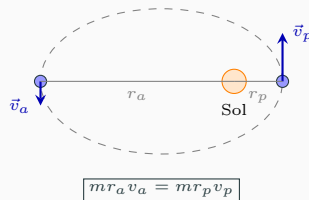
Caso del Sólido Rígido Cuando las distancias entre masas son fijas y rotan respecto a un eje, la expresión se simplifica:

$$\vec{L} = I\vec{\omega}$$

donde I es el momento de inercia del cuerpo.

Ley de Conservación $\vec{L}$ permanece constante incluso con fuerzas externas, siempre que el torque neto sea nulo ($\vec{\tau}_{ext} = \vec{0}$):

Ejemplo: Órbitas Planetarias



La fuerza gravitatoria es central (no genera torque respecto al Sol), por lo que el planeta acelera al acercarse para conservar su $\vec{L}$.

[conservacion_L.py](#)

4 Control de colisiones

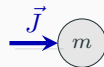
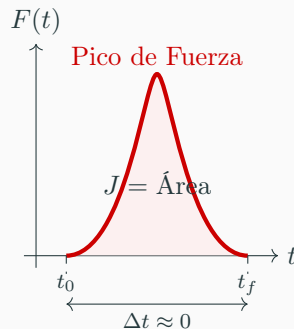
Impulso y Cambio de Momento

Definición Física El impulso ($\vec{J}$) es la medida del efecto de una fuerza $\vec{F}$ durante un intervalo Δt :

$$\vec{J} = \int_{t_0}^{t_0 + \Delta t} \vec{F}(t) dt = \Delta \vec{p}$$

Naturaleza del Choque

- Son fuerzas **impulsivas**: extremas en tiempos mínimos.
- Dependen de la deformación elástica de los materiales.
- Pymunk y PyBullet **omiten la fuerza** y aplican directamente un impulso $\Delta \vec{p}$ (**básicamente un $\Delta \vec{v}$**).
- En un proceso iterativo se van añadiendo pequeños $\Delta \vec{p}$ y se comprueba que los cuerpos no se atraviesan y se aplica la ley de Coulomb de rozamiento.



Aproximación del Impulso y Fuerzas Externas

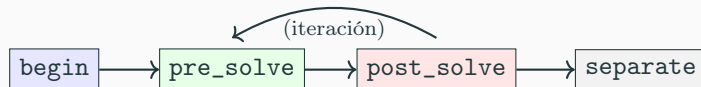
En ocasiones, no nos interesa simular la geometría del contacto, sino únicamente el efecto del intercambio de energía.

- **Aproximación del impulso:** Si el objetivo es transferir momento sin una colisión explícita entre formas (*shapes*), se recurre a modelos de fuerzas externas.
- **Implementación:** Se aplica una fuerza $\vec{F}$ que actúa durante un intervalo de tiempo determinado Δt .
- El cambio en el momento lineal se define como: $\vec{J} = \int \vec{F} dt \simeq \vec{F} \cdot \Delta t = \Delta \vec{p}$
- Esto es útil para simular explosiones cercanas, propulsores o campos de fuerza donde el contacto físico es irrelevante

Control de Colisiones: Callbacks

El motor permite intervenir la física mediante un **Collision Handler**. Existen cuatro eventos donde podemos inyectar código:

- **begin**: Se llama una única vez cuando dos formas se tocan por primera vez.
Retorno: **True** para procesar el choque, **False** para ignorarlo.
- **pre_solve**: Se llama en cada *step* mientras dura el contacto. Permite cambiar parámetros como la fricción o elasticidad justo antes del cálculo.
- **post_solve**: Se llama tras el cálculo físico. Es el momento ideal para leer la fuerza aplicada o destruir objetos (ej. una bala que desaparece).
- **separate**: Se llama cuando los objetos dejan de tocarse.



Puentear el Comportamiento por Defecto

Para "puentear" la física y aplicar impulsos manuales, la estrategia es:

1. Clasificamos los tipos de objetos que van a aparecer en la simulación asignándoles un `self.shape.collision_type` (ej. 1 para la bola blanca, 2 para las bolas de color y 3 para el taco)
2. Definimos un handler para los tipos de colisión:
`space.on.collision(tipo1,tipo2, begin=mi_funcion_begin)` Si es necesario se hace también para `pre_solve`, `post_solve` y/o `separate`
3. Implementar la lógica en `begin` o `pre_solve`.
4. **IMPORTANTE:** Retornar `False`.

Al retornar `False`, el *Solver* de Pymunk descarta el cálculo de fuerzas normal y tangencial, permitiéndonos aplicar nuestro propio impulso sin interferencias.

Ejemplos

- En `linea_accion.py` tenemos un solo tipo de objetos que colisionan (la clase Bola). Sobreescribimos `begin` para sacar por pantalla las velocidades justo antes de la colisión y también `separate` para dejar detenida la simulación y mostrar la línea de acción.
- En `billar_taco.py`. Asignamos `self.shape.collision_type = 1` a las bolas y 2 al taco. Sobreescribimos `post_solve` para dejar el taco parado en seco.
- En `billar_taco_impulso.py` sobreescribimos `begin` para la colisión entre bola y taco y calculamos manualmente el impulso lineal y angular que añadimos a la bola.
IMPORTANTE: devuelve `False` para que no se llame al resto de funciones.